A

Project Report

On

DEEP LEARNING APPROACH FOR AN ANALYSIS OF REAL-ESTATE PRICES & TRANSACTIONS

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted By

R. SINDHU

(22HM1A3331)

D. VENKATESULU

(22HM1A3307)

N. TEJASHWINI

(22HM1A3327)

S. MOHAMMED NIHAL

(22HM1A3340)

K. RAMANJANEYULU

(22HM1A3320)

Under the esteemed guidance of

Mr. S. MOHAMMAD ALI M. Tech.,

Assistant Professor,

Department of CSE (AI&ML).

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Affiliated to J.N.T.U.A., Anantapuramu, Approved by A.I.C.T.E, New Delhi)

Accredited by NAAC with 'A' Grade & NBA (EEE, ECE & CSE)

Utukur (P), C.K. Dinne (V&M), Kadapa-516003

Andhra Pradesh.

2022-2026.

A

Project Report

On

DEEP LEARNING APPROACH FOR AN ANALYSIS OF REAL-ESTATE PRICES & TRANSACTIONS

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted By

R. SINDHU

(22HM1A3331)

D. VENKATESULU

(22HM1A3307)

N. TEJASHWINI

(22HM1A3327)

S. MOHAMMED NIHAL

(22HM1A3340)

K. RAMANJANEYULU

(22HM1A3320)

Under the esteemed guidance of

Mr. S. MOHAMMAD ALI M. Tech.,

Assistant Professor,

Department of CSE (AI&ML).

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Affiliated to J.N.T.U.A., Anantapuramu, Approved by A.I.C.T.E, New Delhi)

Accredited by NAAC with 'A' Grade & NBA (EEE, ECE & CSE)

Utukur (P), C.K. Dinne (V&M), Kadapa-516003

Andhra Pradesh.

2022-2026.

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Affiliated to J.N.T.U.A. Anantapuramu, Approved by A.I.C.T.E, New Delhi)

Accredited by NAAC with 'A' Grade & NBA (EEE, ECE & CSE)

Utukur (P), C.K. Dinne (V&M), Kadapa-516003.

CERTIFICATE

This is to certify that the project work entitled **"DEEP LEARNING APPROACH FOR AN ANALYSIS OF REAL-ESTATE PRICES & TRANSACTIONS"** is a Bonafide work done by

R. SINDHU	**(22HM1A3331)**
D. VENKATESULU	**(22HM1A3307)**
N. TEJASHWINI	**(22HM1A3327)**
S. MOHAMMED NIHAL	**(22HM1A3340)**
K. RAMANJANEYULU	**(22HM1A3320)**

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)** in **ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES, KADAPA.** during the academic year 2022-2026. The results of this work have not been submitted to any other university or institutes for the award of any degree.

Project Guide

Mr. S. MOHAMMAD ALI M.Tech.,

Assistant Professor,

Department of CSE (AI&ML)

A.I.T.S., Kadapa.

Head of the Department

Dr. O. HOMA KESAV M.Tech, Ph.D.,

Associate Professor,

Department of CSE (AI&ML)

A.I.T.S., Kadapa.

Date:

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project report entitled **"DEEP LEARNING APPROACH FOR AN ANALYSIS OF REAL-ESTATE PRICES & TRANSACTIONS"** is a record of project work carried out by us for award of the degree of Bachelor of Technology in CSE (Artificial Intelligence & Machine Learning). We also declare that this project is a result of our own effort and has not been submitted earlier for the award of any Degree or other courses or any other University.

PROJECT ASSOCIATES

R. SINDHU	**(22HM1A3331)**
D. VENKATESULU	**(22HM1A3307)**
N. TEJASHWINI	**(22HM1A3327)**
S. MOHAMMED NIHAL	**(22HM1A3340)**
K. RAMANJANEYULU	**(22HM1A3320)**

ACKNOWLEDGEMENT

We are deeply indebted to our supervisor **Mr. S. MOHAMMAD ALI, Assistant Professor, Dept. of CSE (AI&ML)** for his valuable guidance, constant encouragement, constructive criticism and keen interest evinced throughout the course of our work. We are really fortunate to associate ourselves with such an advising and helping guide in every possible way, at all stages, for the successful completion of this project work.

We extend our gratefulness to our Coordinator, **Mr. S. MOHAMMAD ALI, Assistant Professor, Dept. of CSE (AI&ML)** for his encouragement and support throughout the project.

We are extremely thankful to **Dr. O. HOMA KESAV**, **HEAD OF THE DEPARTMENT** of CSE (AI&ML), "Annamacharya Institute of Technology & Sciences" for assisting us in completion of this project.

We express our gratitude to our principal **Dr. A. SUDHAKARA REDDY** and the **Management** for providing all the facilities and supporting in completing our Project work successfully.

We express our heartful thanks to entire **Faculty Members** in the department of **CSE (Artificial Intelligence & Machine Learning)** of Annamacharya Institute of Technology & Sciences, for their moral support and good wishes.

Last, but least by any means, we are thankful to all the non-teaching staff members of **CSE (Artificial Intelligence & Machine Learning)** Department for their extended co-operation.

PROJECT ASSOCIATES

R. SINDHU (22HM1A3331)
D. VENKATESULU (22HM1A3307)
N. TEJASHWINI (22HM1A3327)
S. MOHAMMED NIHAL (22HM1A3340)
K. RAMANJANEYULU (22HM1A3320)

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

VISION OF THE DEPARTMENT

- To Achieve excellent standards of quality education by using the latest tools, nurturing collaborative culture and disseminating customer-oriented innovations to relevant areas of academia and industry towards serving the greater cause of society.

MISSION OF THE DEPARTMENT

- To develop professionals who are skilled in the area of CSE (Artificial Intelligence & Machine Learning).
- To impart quality and value-based education and contribute towards the innovation of computing expert system, machine learning to raise satisfaction level of all stake holders.
- Our effort is to apply new advancements in high performance computing hardware and software.

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Mapping of PO's through Project

Title of the project	**Deep Learning Approach for an Analysis of Real-Estate Prices & Transactions**	
Students	**Roll Number**	**Name of the Student**
	22HM1A3331	R. SINDHU
	22HM1A3307	D. VENKATESULU
	22HM1A3327	N. TEJASHWINI
	22HM1A3340	S. MOHAMMED NIHAL
	22HM1A3320	K. RAMANJANEYULU

SYNOPSIS:

This project presents a real-time Real Estate Price Prediction System that helps users estimate land prices and construction costs efficiently. It uses Deep Learning techniques with an Artificial Neural Network (ANN) model to analyze property features such as location, area, and property specifications for accurate predictions. The system includes data preprocessing to handle and transform input data for better model performance. It supports both land price prediction and construction cost estimation based on user inputs. A web-based interface developed using Django allows users to easily interact with the system and obtain instant results. The system also maintains prediction history and provides real estate agent contact details for user assistance. With features like fast prediction, user-friendly interface, and data-driven analysis, the system offers a scalable and reliable solution for real-world real estate decision-making.

Title of the project	**PO 1**	**PO 2**	**PO 3**	**PO 4**	**PO 5**	**PO 6**	**PO 7**	**PO 8**	**PO 9**	**PO 10**	**PO 11**	**PO 12**
Deep Learning Approach for an Analysis of Real-Estate Prices & Transactions	**3**	**3**	**3**	**2**	**3**	**2**	**3**	**2**	**3**	**2**	**2**	**3**

Abstract

The rapid growth of urbanization and increasing demand for residential and commercial properties has made accurate real estate price estimation very important. Traditional property valuation methods mainly depend on manual analysis and broker opinions, which may lead to inconsistent and unreliable price estimates. To address this issue, this project proposes a Deep Learning–based Real Estate Price and Construction Cost Prediction System using Artificial Neural Networks (ANN). The system focuses on two main prediction modules: land price prediction and construction cost prediction. Land price prediction considers factors such as state, district, land area, plot type, soil type, and land characteristics to understand how geographical and environmental conditions affect land value. Construction cost prediction uses parameters like built-up area, number of bedrooms, bathrooms, and age of the property to estimate building costs accurately. A web-based application is developed using the Django framework to allow users to enter property details and receive predictions instantly. The system includes features such as dynamic state–district selection, secure user authentication, prediction history storage, and dashboard visualization. The ANN model is trained using processed datasets to provide accurate, data-driven, and unbiased predictions. The system is also designed to be scalable and can be extended in the future by integrating real-time market data, GIS-based spatial analysis, and government land records. By providing fast and intelligent predictions, the system reduces dependence on manual valuation methods, minimizes pricing errors, and helps buyers, investors, and builders make better decisions in the real estate market.

TABLE OF CONTENTS

CHAPTER NO.	CHAPTER NAME	PAGE NO.
List of figures		i-ii
List of Abbreviations		iii
CHAPTER-1	**INTRODUCTION**	**1-2**
1.1	Introduction	1
1.2	Project Objectives	2
CHAPTER-2	**SYSTEM ANALYSIS**	**3-5**
2.1	Existing System	3
	2.1.1 Disadvantages	3
2.2	Proposed System	4
	2.2.1 Advantages	5
CHAPTER-3	**SOFTWARE REQUIREMENTS & SPECIFICATIONS**	**6-8**
3.1	Hardware Requirements	6
3.2	Software Requirements	6
3.3	Software Description	6
	3.3.1 Python	6-7
	3.3.2 History of Python	7
	3.3.3 Python Features	7
	3.3.4 Feasibility Study	8
CHAPTER-4	**SYSTEM DESIGN**	**9-22**
4.1	System Architecture	9-10
4.2	Modules Description	10-12
	4.2.1 User Authentication	11
	4.2.2 Data Preprocessing	11
	4.2.3 Model Training	12
	4.2.4 Price Prediction	12

	4.2.5 History & Dashboard	12
4.3	Introduction to UML	13
4.4	UML Diagrams	13
4.5	Class Diagram	14-15
4.6	Use Case Diagram	15-17
4.7	Sequence Diagram	18-19
4.8	Activity Diagram	20-22
CHAPTER-5	**SYSTEM CODING & IMPLEMENTATION**	**23-41**
5.1	Source Code	23-36
5.2	System Testing	37-41
	5.2.1 Testing Methodologies	37-39
	5.2.2 Testing Strategy	39-40
	5.2.3 Types of Tests	40-41
	5.2.3.1 Unit Testing	40
	5.2.3.2 Integration Testing	40
	5.2.3.3 System Testing	41
	5.2.3.4 Functional Testing	41
	5.2.3.5 User Acceptance Testing	41
CHAPTER-6	**EXECUTION STEPS**	**42-47**
6.1	Installation of Software	42-44
6.2	Installation of Django	45-46
6.3	Installation of Libraries	47
CHAPTER-7	**SCREENSHOTS**	**48-60**
CHAPTER-8	**CONCLUSION**	**61**
CHAPTER-9	**FUTURE ENHANCEMENTS**	**62**
CHAPTER-10	**BIBLIOGRAPHY**	**63**

LIST OF FIGURES

S.NO.	FIGURE NAME	FIG NO.	PAGE NO.
1.	System Architecture	4.1	9
2.	System Modules	4.2	11
3.	Class Diagram	4.3	14
4.	Use Case Diagram	4.4	16
5.	Sequence Diagram	4.5	19
6.	Activity Diagram	4.6	21
7.	Python Verification	6.1	42
8.	Download Python	6.2	43
9.	Python Installing	6.3	43
10.	Python Installation Process	6.4	44
11.	Installation Successful	6.5	44
12.	Django Installation	6.2.1	45
13.	Django Version	6.2.2	45
14.	Running Django Server	6.2.3	46
15.	Django Welcome Page	6.2.4	46
16.	Running the Project	7.1	48
17.	Welcome Page	7.2	49
18.	User Signup Page	7.3	50
19.	User Login Page	7.4	50
20.	Home Page	7.5	51

S.NO.	FIGURE NAME	FIG NO.	PAGE NO.
21.	Land Prediction Input Page	7.6	52
22.	Land Prediction Result Page	7.7	52
23.	Land Prediction History page	7.8	53
24.	Land Prediction Dashboard	7.9	54
25.	Construction Prediction Input Page	7.10	55
26.	Construction Prediction Result Page	7.11	55
27.	Construction Prediction History Page	7.12	56
28.	Construction Prediction Dashboard	7.13	57
29.	Admin Login Page	7.14	57
30.	Admin Home Page	7.15	58
31.	Add Realtor Page	7.16	59
32.	Realtors Page	7.17	59

LIST OF ABBREVIATIONS

- **DL** - Deep Learning
- **AI** - Artificial Intelligence
- **ANN** - Artificial Neural Network
- **API** - Application Programming Interface
- **GPS** - Global Positioning System
- **GIS** - Geographic Information System
- **UI** - User Interface
- **DB** - Database
- **URL** - Uniform Resource Locater
- **HTTP** - Hypertext Transfer Protocol

Abstract

The rapid growth of urbanization and increasing demand for residential and commercial properties has made accurate real estate price estimation very important. Traditional property valuation methods mainly depend on manual analysis and broker opinions, which may lead to inconsistent and unreliable price estimates. To address this issue, this project proposes a Deep Learning–based Real Estate Price and Construction Cost Prediction System using Artificial Neural Networks (ANN). The system focuses on two main prediction modules: land price prediction and construction cost prediction. Land price prediction considers factors such as state, district, land area, plot type, soil type, and land characteristics to understand how geographical and environmental conditions affect land value. Construction cost prediction uses parameters like built-up area, number of bedrooms, bathrooms, and age of the property to estimate building costs accurately. A web-based application is developed using the Django framework to allow users to enter property details and receive predictions instantly. The system includes features such as dynamic state–district selection, secure user authentication, prediction history storage, and dashboard visualization. The ANN model is trained using processed datasets to provide accurate, data-driven, and unbiased predictions. The system is also designed to be scalable and can be extended in the future by integrating real-time market data, GIS-based spatial analysis, and government land records. By providing fast and intelligent predictions, the system reduces dependence on manual valuation methods, minimizes pricing errors, and helps buyers, investors, and builders make better decisions in the real estate market.

CHAPTER 1
INTRODUCTION

1. INTRODUCTION

1.1 INTRODUCTION

Real estate price estimation is an important task for buyers, investors, and property developers. Property prices are influenced by many factors such as location, demand, infrastructure development, and property features. Due to the rapid growth of urbanization and increasing demand for residential and commercial properties, accurate price prediction has become very important in the real estate sector. Traditional methods of property valuation mainly rely on manual calculations, market analysis, and real estate agent opinions. These methods are often subjective and may lead to inaccurate or biased price estimates. As a result, buyers and investors may face difficulties in making proper investment decisions.

To overcome these limitations, this project introduces a Deep Learning–based Real Estate Price and Construction Cost Prediction System. The system uses Artificial Neural Networks (ANN) to analyze historical real estate data and predict property prices more accurately. The system focuses on two major predictions: land price prediction and construction cost prediction. Land price prediction is based on location factors such as state, district, and land characteristics. Construction cost prediction considers important building features like built-up area, number of bedrooms, bathrooms, and age of the property.

A web-based application is developed using the Django framework, where users can enter property details and receive predicted prices instantly. The system also provides real estate agent contact details, allowing users to easily connect with professionals for further assistance. By using deep learning techniques and historical data, the system provides reliable, fast, and data-driven price estimation, helping users make better real estate decisions.

The diagram illustrates the relationship between land, construction, and price in the real estate prediction system. Land represents the value of the plot based on factors

such as location and area. Construction represents the cost required to build a property, which depends on parameters like built-up area, number of bedrooms, bathrooms, and age of the building. Both land value and construction cost play an important role in determining the overall property price. The system analyzes these two components separately using prediction models.

The land prediction module estimates the price of the land, while the construction prediction module calculates the cost of building the structure. These two values together help determine the final estimated price of the property. The arrows in the diagram indicate that both land and construction irectly influence the price. This approach provides a clear understanding of how the total property value is calculated. It helps users make better decisions regarding real estate investments and construction planning.

1.2 PROJECT OBJECTIVES

Here are simple project objectives you can use:

- To develop a real estate price prediction system using deep learning techniques.
- To predict land prices based on location factors such as state, district, and plot details.
- To estimate construction costs based on property features like area, BHK, bathrooms, and age of the building.
- To design a user-friendly web application using Django for entering property details and viewing predictions.
- To provide accurate and data-driven price estimation using historical real estate data.
- To store and manage user prediction history and system data through the admin panel.
- To provide real estate agent contact details to help users connect with professionals.
- To reduce dependency on manual valuation and broker opinions in real estate pricing.

CHAPTER 2
SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The existing system for real estate analysis mainly relies on traditional statistical methods and manual data collection techniques. In many cases, researchers use questionnaire-based surveys and basic statistical analysis to study the real estate market. These methods are mainly used to identify the important factors that influence property prices and real estate transactions. Factors such **as** economic conditions, population growth, construction costs, and market demand are usually considered during the analysis.

The existing approach focuses on understanding how different factors affect real estate price changes and market trends. Instead of predicting the exact price of a property, the system mainly analyzes historical data to identify patterns and relationships between variables. Researchers study these patterns to understand why property prices increase or decrease over time.

However, this traditional system has several limitations. The analysis process is time-consuming and requires significant manual effort. It is also not suitable for handling large and complex datasets, which are common in modern real estate markets. Because of these limitations, the accuracy and efficiency of the analysis may be reduced. Therefore, there is a need for more advanced systems that can automatically process large datasets and provide better insights into real estate price trends.

2.1.1 DISADVANTAGES

- Real estate price prediction is not a feature of this platform. The existing system focuses only on analyzing historical real estate data rather than generating price predictions. It studies past market behavior to understand price trends. Therefore, it cannot estimate the price of a specific property for users.

- It analyzes real estate price changes and identifies influencing factors. The system examines various economic and demographic factors that affect real estate prices. These may include population growth, construction costs, and market demand. The goal is to understand why prices increase or decrease over time.

- Agent details are not included or used in the analysis. The study focuses purely on data analysis related to property prices and market trends. It does not include information about real estate agents or agencies. As a result, users cannot find or contact agents through this system.

- It is only a research study and does not provide any user interface or application. The system was developed mainly for academic research purposes. There is no web application or software interface where users can interact with the system. Only researchers use the model to analyze data and study results.

- The analysis is focused only on houses and residential properties.The dataset and analysis concentrate mainly on residential real estate such as houses and apartments. Other property types like land, commercial buildings, or industrial properties are not included. Therefore, the results are limited to residential market analysis.

2.2 PROPOSED SYSTEM

The proposed system is a web-based real estate price prediction platform designed to estimate land prices and construction costs using machine learning techniques. The system helps users obtain reliable property price estimates based on various location and property-related parameters. The system uses Artificial Neural Networks (ANN) to analyze historical real estate data and identify patterns that influence property values. By using deep learning techniques, the system can generate accurate and data-driven price predictions.Users can access the platform through a web interface developed using the Django framework**.** The interface allows users to enter important property details such as state, district, land area, plot type, number of bedrooms, bathrooms, and age of the property. Based on these inputs, the system predicts the estimated land price or construction cost. The system includes two main prediction modules: land price prediction and construction cost prediction. Land price prediction focuses on location-based factors, while construction cost prediction considers structural and architectural parameters. The platform also includes secure user authentication, allowing users to register and log in to access the system. Once logged in, users can perform predictions and view their prediction history through the dashboard.

All prediction results and user activities are stored in a database and managed through an admin panel**.** The admin has full control to monitor user activities, manage prediction records, and maintain system data. Another important feature of the system is the real estate agent contact module**.** Agent details are stored and maintained by the admin, and users can view these contact details to connect with real estate professionals for further assistance. By integrating automated price prediction with agent contact information in a single platform, the system improves accuracy, transparency, and user decision-making in the real mark

2.2.1 ADVANTAGES

- The system automatically predicts property prices using an ANN model. The system uses an Artificial Neural Network (ANN) to analyze property features and estimate the price automatically. This helps provide faster and more accurate price predictions compared to manual methods.
- It provides a user-friendly interface through a Django-based web application. The system includes a web interface built using Django where users can easily enter property details. It allows users to interact with the system and get results without technical knowledge.
- The system can handle both land and construction-related data. The model is designed to process information related to land as well as constructed properties. This makes the system more flexible for different types of real estate predictions.
- It also provides agent details based on the selected location. The system displays contact details of nearby real estate agents according to the selected area. This helps users easily connect with professionals for buying or selling properties.
- Easy to maintain and update through the admin panel. The admin panel allows administrators to manage datasets, users, and agent details efficiently. It also makes it easy to update system information without modifying the core concepts.

CHAPTER 3

SOFTWARE REQUIREMENTS & SPECIFICATIONS

3. SOFTWARE REQUIREMENTS & SPECIFICATIONS

3.1 HARDWARE REQUIREMNTS

Processor	:	Intel i3 or above
RAM	:	Minimum 4 GB
Hard Disk	:	10 GB free space

3.2 SOFTWARE REQUIREMENTS

Operating System	:	Windows
Programming Language	:	Python 3.x
Framework	:	Django
Frontend	:	HTML, CSS, JavaScript
Database	:	SQLite
Web Browser	:	Chrome
IDE	:	VS Code

3.3 SOFTWARE DESCRIPTION

Python is a high-level, interpreted programming language known for its simplicity and readability. It supports multiple programming paradigms including object-oriented, procedural, and functional programming. Python is widely used in fields such as web development, data science, machine learning, and artificial intelligence. Due to its extensive libraries and frameworks, Python is considered one of the most suitable languages for developing machine learning applications and web-based systems.

3.3.1 INTRODUCTION TO PYTHON

Python is a popular programming language that is easy to learn and widely used for software development. It provides a simple syntax that allows developers to write clear and efficient code. Python is highly versatile and is used in various domains including machine learning, artificial intelligence, web development, and data analysis.

In this project, Python is used to implement the Artificial Neural Network (ANN) model and handle the prediction logic for estimating real estate prices.hich help developers build advanced applications .

3.3.2 HISTORY OF PYTHON

Python was created by **Guido van Rossum** and first released in **1991**. The language was designed with the goal of improving code readability and simplifying software development. Python was influenced by several programming languages such as ABC, C, and Modula-3.

Over the years, Python has evolved significantly with several updated versions introducing new features and improvements. Python 2 was released in 2000 and gained widespread popularity. Later, Python 3 was introduced in 2008 with improved performance and better support for modern programming practices. Today, Python is one of the most widely used programming languages in the world, especially in the fields of machine learning, artificial intelligence, and web application development.

3.3.3 FEATURES OF PYTHON

- **Simple and Easy to Learn** – Python has a clean and easy-to-understand syntax, making it suitable for beginners and developers.
- **Platform Independent** – Python programs can run on multiple operating systems such as Windows, Linux, and macOS without modification.
- **Extensive Libraries** – Python provides a large collection of libraries and frameworks for machine learning, data analysis, and web development.
- **Object-Oriented Programming** – Python supports object-oriented concepts such as classes and objects, enabling developers to create modular and reusable code.
- **Integration with Machine Learning** – Python supports many machine learning libraries that help build predictive models such as Artificial Neural Networks.
- **Web Development Support** – Python frameworks like **Django** allow developers to build secure and scalable web applications.
- **Dynamic Typing** – Python automatically handles data types, which simplifies coding and reduces development time.

3.3.4 FEASIBILITY STUDY

An important outcome of preliminary investigation is the determination that the system request is feasible. The different feasibilities that have to be analyzed are

- Operational Feasibility
- Economic Feasibility
- Technical Feasibility

Operational Feasibility

Operational feasibility refers to how well the proposed system will work in the real environment and whether users will accept it. The proposed system provides a user-friendly web interface through Django where users can easily enter property details and get predicted prices. It also helps users find real estate agent details based on location, making the system useful for buyers and sellers.

Economic Feasibility

Economic feasibility evaluates the cost required to develop and maintain the system. The proposed system uses open-source technologies such as Python, Django, and machine learning libraries, which reduces development costs. Since the system can run on standard computers without requiring expensive hardware, it is economically feasible.

Technical Feasibility

Technical feasibility determines whether the available technology and resources are sufficient to build the system. The proposed system uses Artificial Neural Networks (ANN) for price prediction and Django for web development, both of which are widely used and reliable technologies. The required tools, libraries, and development environment are easily available, making the system technically feasible.

CHAPTER 4

SYSTEM DESIGN

4.SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The architecture diagram represents the working flow of the Real Estate Price Prediction System based on Deep Learning. The process begins with the collection of data, which includes historical real estate information such as land details and construction-related parameters. This data forms the foundation for building an accurate prediction system. Once the data is collected, it undergoes a preprocessing stage. Data preprocessing is an important step where raw data is cleaned, organized, and transformed into a suitable format for model training. This includes handling missing values, encoding categorical variables like state and district, and normalizing numerical features such as area and number of rooms.

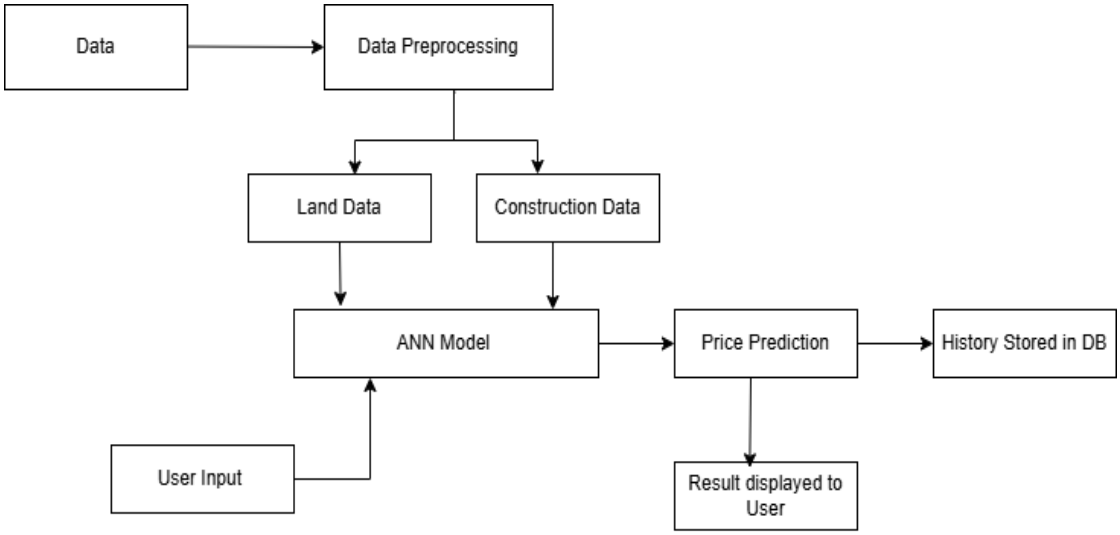


Fig.4.1: System Architecture

After preprocessing, the data is divided into two categories: land data and construction data. Land data includes features like location, plot type, and land area, while construction data includes features such as built-up area, number of bedrooms, bathrooms, and property age. These two types of data are used separately for different prediction purposes. After preprocessing, the data is divided into two categories: land data and construction data. Land data includes features like location, plot type, and land area,

while construction data includes features such as built-up area, number of bedrooms, bathrooms, and property age. These two types of data are used separately for different prediction purposes.

At the same time, the system also takes input from the user through the web interface. The user provides details such as location, size, and other property-related information. This user input is processed and combined with the trained ANN model for prediction. Based on the input provided, the ANN model performs price prediction. It analyzes the input features and generates an estimated price for either land or construction. The predicted price is then forwarded to the next stage of the system.

The prediction result is stored in the database for future reference. This helps in maintaining user history and allows users to view their past predictions. It also enables the admin to monitor system usage and manage data effectively. Finally, the predicted result is displayed to the user through the web interface. The user can view the estimated price instantly, which helps in making informed real estate decisions.

Overall, this architecture ensures a smooth and systematic flow from data collection to prediction and final result display. It effectively integrates data preprocessing techniques to clean and prepare the input data for accurate analysis. The use of deep learning through the ANN model enables the system to learn complex patterns and relationships in real estate data. The system also incorporates user input through a web interface, making it interactive and easy to use. Predicted results are stored in the database, allowing users to access their history and enabling better data management.

4.2 MODULES DESCRIPTION

The system consists of several modules that work together to perform real estate price prediction. The User Authentication module manages user registration and login for secure system access. The Data Processing and Model Training modules prepare the data and train the ANN model using historical real estate data. The Price Prediction and History & Dashboard modules generate predicted prices and store results for users to view later.

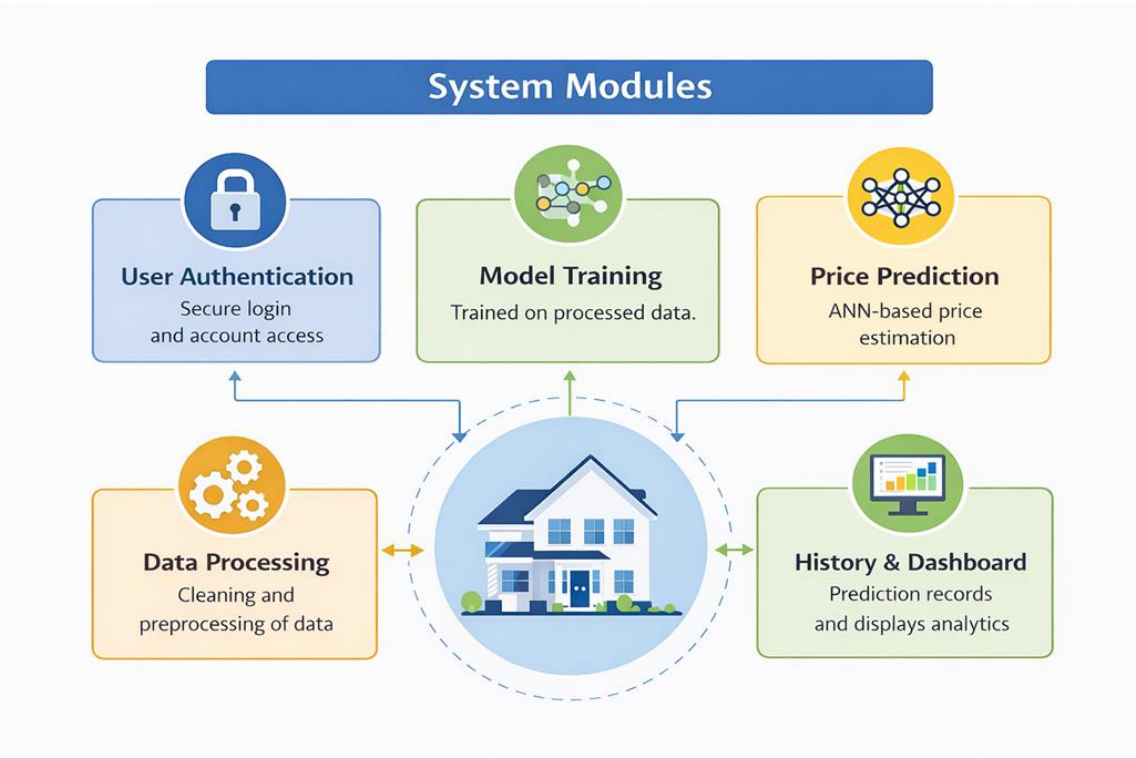


Fig.4.2: System Modules

4.2.1 USER AUTHENTICATION

User authentication is an important module that ensures only authorized users can access the system. In this module, users must first register by creating an account with their details such as username, email, and password. After registration, users can log in using their credentials to access the system features. The authentication system is implemented using the Django authentication framework, which provides secure password hashing and session management. This ensures that user passwords are stored safely and cannot be easily accessed by unauthorized users. Once logged in, users can securely interact with the system, perform price predictions, and view their prediction history. This module helps maintain system security and protects user information.

4.2.2 DATA PROCESSING

The data processing module prepares the input data before it is used by the prediction model. Raw data collected from users or datasets may contain different types of values, including categorical and numerical data. Categorical data such as state, district, and city cannot be directly used by machine learning models, so they are

converted into numerical form using encoding techniques. Numerical features such as land area, number of bedrooms, bathrooms, and property age are normalized or scaled so that the model can process them efficiently. Data processing also ensures that the input data is clean, consistent, and suitable for machine learning algorithms.

4.2.3 MODEL TRAINING

The model training module is responsible for building and training the Artificial Neural Network (ANN) used for price prediction. In this module, the processed dataset containing historical property information is used to train the model. The ANN learns patterns and relationships between property features and price values through multiple training iterations. The network consists of an input layer, hidden layers, and an output layer that produces the predicted price. During training, the model adjusts its internal weights to minimize prediction errors. After the training process is completed, the trained model is saved so that it can be reused for making predictions without retraining each time.

4.2.4 PRICE PREDICTION

The price prediction module is the main functionality of the system. In this module, users enter property details such as state, city, plot type, land area, number of bedrooms, bathrooms, and age of the property through the web interface. These inputs are first processed and then passed to the trained ANN model. The model analyzes the input values and predicts the estimated land price or construction cost based on patterns learned from historical data. The prediction is generated instantly and displayed to the user on the screen. This module helps users quickly estimate property values without relying on manual calculations or broker opinions.

4.2.5 HISTORY & DASHBOARD

The history and dashboard module manages the storage and visualization of prediction results. Every time a user performs a prediction, the result along with the input details is stored in the system database. Users can view their previous predictions in the history section, which helps them track and compare property values. The dashboard provides a summarized view of prediction data and may include graphical insights or statistics. This module improves user experience by allowing users to easily access past

predictions and analyze property trends. It also helps the admin monitor system usage and manage prediction records efficiently.

4.3 INTRODUCTION TO UML

Unified Modeling Language (UML) is a standardized visual language used to design and represent software systems. In this project, UML is used to model the Real Estate Price and Construction Cost Prediction System and to show how different parts of the system interact.

UML diagrams such as Use Case Diagrams, Activity Diagrams, and Sequence Diagrams help illustrate user interactions, system processes, and prediction workflows. These diagrams represent components like user authentication, data processing, price prediction, and result display. Using UML makes the system design easier to understand, improves communication, and clearly explains how the real estate prediction system works.

4.4 UML DIAGRAMS

Unified Modeling Language (UML) diagrams are visual representations used to design and describe the structure and behavior of a software system. In this project, UML diagrams are used to represent the architecture and workflow of the Real Estate Price and Construction Cost Prediction System. These diagrams help developers and stakeholders clearly understand how different components of the system interact.

UML diagrams illustrate important system functionalities such as user authentication, data processing, ANN-based price prediction, and result display. They also show how users interact with the system to enter property details and obtain predicted land prices and construction costs. By visualizing the system processes, UML diagrams make the system easier to analyze, design, and implement. In this project, UML diagrams such as Use Case Diagrams, Activity Diagrams, and Sequence Diagrams are used to explain system operations and workflows. These diagrams help identify system components, their relationships, and the flow of information between different module.

4.5 CLASS DIAGRAM

The UML Class Diagram represents the structure of the Real Estate Price Prediction System by showing the main classes, their attributes, methods, and relationships. A class diagram helps to describe how different parts of the system are organized and how they interact with each other.

The diagram contains four main classes: User, Property Query, Land Query, and Realtor. Each class represents an important component of the system and contains attributes (data) and methods (functions). The User class represents the system users who access the application.

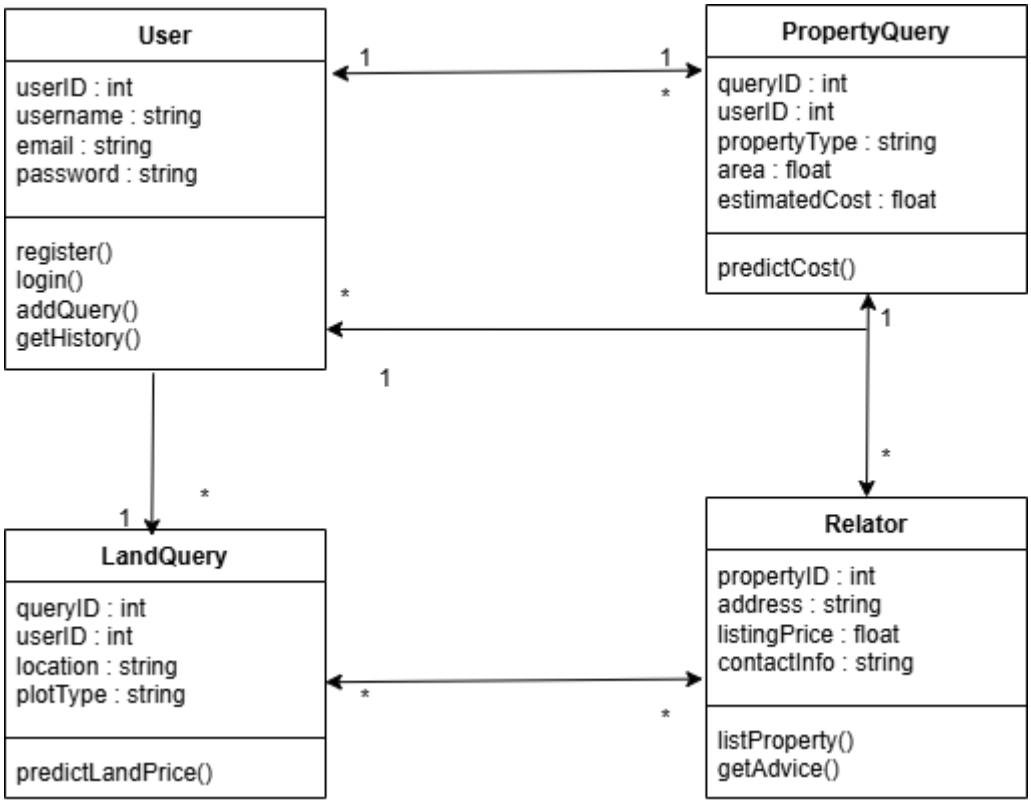


Fig.4.3: Class Diagram

This class contains attributes such as userID, username, email, and password, which store user information. The methods in this class include register(), login(), addQuery(), and getHistory(). These methods allow users to create an account, log in to the system, submit property queries for prediction, and view their previous prediction history. The Propert

Query class represents the property-related queries submitted by users for prediction. It contains attributes such as query ID, user ID, property Type, area, These attributes store the details of the property and the predicted and estimated Cost. cost generated by the system. The method predict Cost() is used to calculate the predicted cost based on the property information provided by the user.

The Land Query class represents queries related specifically to land price prediction. This class includes attributes such as query ID, user ID, location, and plot Type, which describe the land details entered by the user. The method predict Land Price() is used to estimate the land price. The Realtor class represents real estate agents who provide property information and assistance to users. This class contains attributes such as property ID, address, listing Price, and contact Info, which store the details of properties and agent contact information. The methods listProperty() and getAdvice() allow realtors to list properties and provide guidance or support to users.

The relationships between these classes show how they interact. A User can create multiple PropertyQuery and LandQuery records, which means one user can perform many prediction queries. This represents a one to-many relationship - between the User class and the query classes. The PropertyQuery class is also connected to the Realtor class, indicating that property predictions may be associated with realtor information. Similarly, the LandQuery class can interact with the Realtor class to provide property listings or agent contacts related to the predicted location.

Overall, this class diagram illustrates the core structure of the system, showing how users interact with property and land queries, how predictions are generated, and how realtor information is integrated into the system to assist users in making better real estate decisions.

4.6 USE CASE DIAGRAM

The Use Case Diagram represents the interaction between the actors and the Real Estate Price Prediction System. It illustrates how different users interact with the system and what operations they can perform. In this system, there are two main actors: Admin and User. These actors interact with different functions of the application to perform various tasks related to property price prediction and system management. Both the

Admin and the User must first access the system through the Register/Login functionality. New users can create an account by registering with their details, while existing users can log in using their credentials.

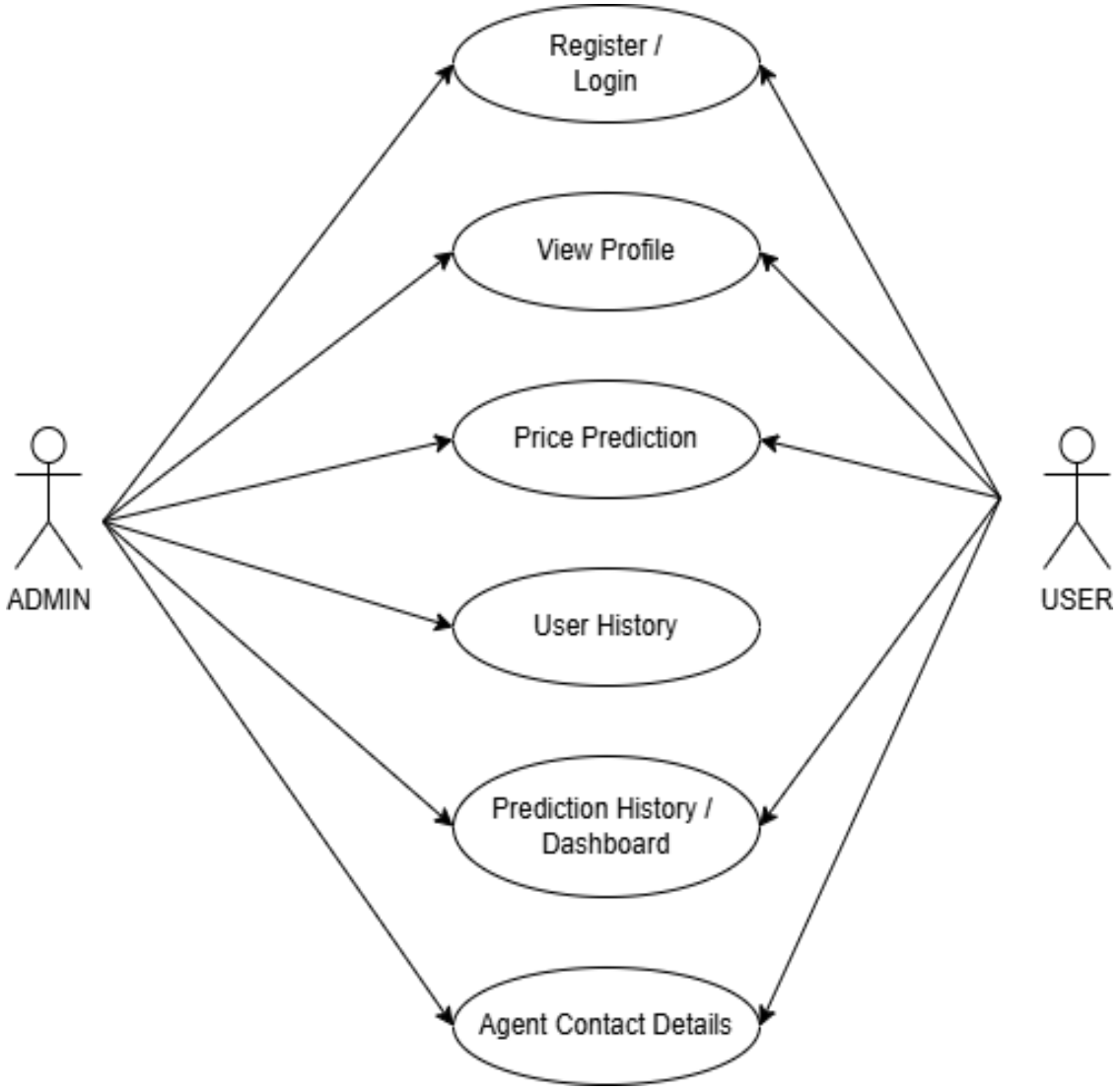


Fig.4.4: Use Case Diagram

This authentication process ensures that only authorized users can access the system and its features. After successfully logging in, both the admin and the user can view their profile information. The profile section allows them to see their account details and manage their information within the system. This helps maintain user records and provides personalized access to the platform.

One of the main functionalities of the system is Price Prediction. Both the admin and the user can perform price predictions by entering required property details such as location, plot type, land area, or building features. The system processes the provided input data and uses the trained Artificial Neural Network(ANN) model to generate predicted land prices or construction costs. All prediction activities performed by users are stored in the User History module.

This module keeps track of the prediction inputs and results for each user. The admin is responsible for managing and maintaining this history data to ensure that records are properly stored and organized. The system also provides a Prediction History and Dashboard feature. Users can view their previous predictions and analyze their past activities through the dashboard. This section helps users keep track of their prediction results and provides useful insights. The admin has full control over this module and can monitor, manage, and maintain all stored data.

Another important feature of the system is the Agent Contact Details module. This module provides users with contact information of real estate agents who can assist them with property-related services. Users can view the agent details such as name, phone number, and location to connect with professionals for further guidance. However, users only have permission to view the agent contact details, while the admin has the responsibility to add, update, and maintain the agent information in the system. This ensures that the information provided to users is accurate and up to date.

The system also provides a Prediction History and Dashboard feature. Users can view their previous predictions and analyze their past activities through the dashboard. This section helps users keep track of their prediction results and provides useful insights. The admin has full control over this module and can monitor, manage, and maintain all stored data. Users can view the agent details such as name, phone number, and location to connect with professionals for further guidance. However, users only have permission to view the agent contact details, while the admin has the responsibility to add, update, and maintain the agent information in the system.

4.7 SEQUENCE DIAGRAM

The sequence diagram represents the step by step interaction between different components of the Real-estate Price prediction system over time. It shows how message are exchanged between the user, web interface, backend server, and the prediction models to generate the final result.

The diagram includes five main components: User, Web Interface, Backend Server, Land ANN Model, and Construction ANN Model. Each component is represented by a vertical dashed line called a lifeline, which shows the time sequence of operations.

The process begins when the User accesses the system and performs Login or Signup through the web interface. The user sends login or registration information to the Web Interface, which then forwards an authentication request to the Backend Server. The backend server verifies the user credentials. If the authentication is successful, the server sends an authentication success response back to the web interface, allowing the user to access the system. After successful login, the Web Interface displays the property type selection form to the user. The user then selects the type of property for prediction, which can be either land property prediction or construction cost prediction**.**

If the user selects land property prediction, the user enters details related to the land such as location, plot type, and land area. These details are sent from the Web Interface to the Backend Server. The backend server processes this input and sends the required data to the Land ANN Model for prediction. The Land ANN Model analyzes the price.

After the prediction is completed, the model returns the predicted land price to the Backend Server. The backend server then sends the prediction result back to the Web Interface, which displays the result to the user. Similarly, if the user selects construction cost prediction, the user enters construction-related details such as built-up area, number of bedrooms, number of bathrooms, and age of the property.

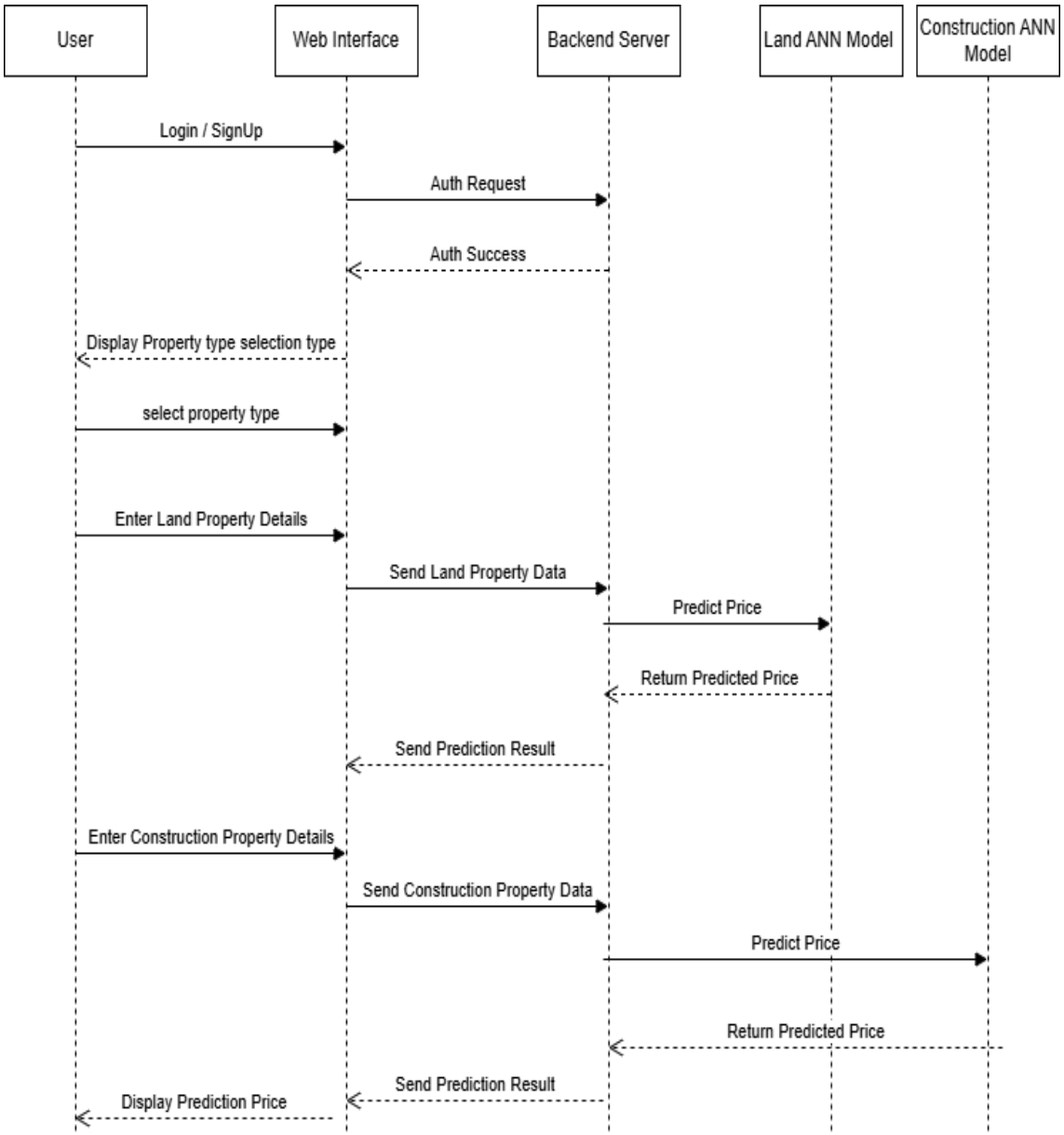


Fig.4.5: Sequence Diagram

These details are sent from the Web Interface to the Backend Server. The backend server forwards the data to the Construction ANN Model. This model uses the trained neural network to estimate the construction cost based on the given inputs. Once the prediction is generated, the model sends the predicted construction cost back to the

Backend Server. The backend server then sends the prediction result to the Web Interface, where it is displayed to the user.

The sequence diagram clearly illustrates how the system processes user input, interacts with prediction models, and returns results in an organized sequence. It highlights the communication between the user interface, backend system, and machine learning models, ensuring that the price prediction process is performed efficiently and accurately.

4.8 ACTIVITY DIAGRAM

The Activity Diagram illustrates the workflow of the Real Estate Price and Construction Cost Prediction System. It shows the sequence of activities performed in the system from the start of the process until the prediction result is displayed to the user. This diagram helps in understanding how the system operates step by step. The process begins with the Start node, which indicates the beginning of the system operation. The first activity is User Login / Signup, where the user either registers a new account or logs into the system using existing credentials. This step ensures that only authenticated users can access the system features.

After successful login, the user proceeds to the next step, which is Enter Property Details. In this stage, the user provides necessary information about the property such as location, plot type, land area, number of bedrooms, bathrooms, or other relevant details required for price prediction. Next, the system reaches a decision point labeled “Property Type?”. At this stage, the system determines whether the user wants to predict the land price or the construction cost. Based on the user’s selection, the process follows one of two paths.

If the user selects Land, the system moves to the Land Prediction process. In this step, the system prepares the input data related to land properties. The system then executes the Run Land ANN Model, where the Artificial Neural Network trained for land price prediction processes the input data and generates a predicted value. If the user selects Construction, the system moves to the Construction Prediction process.

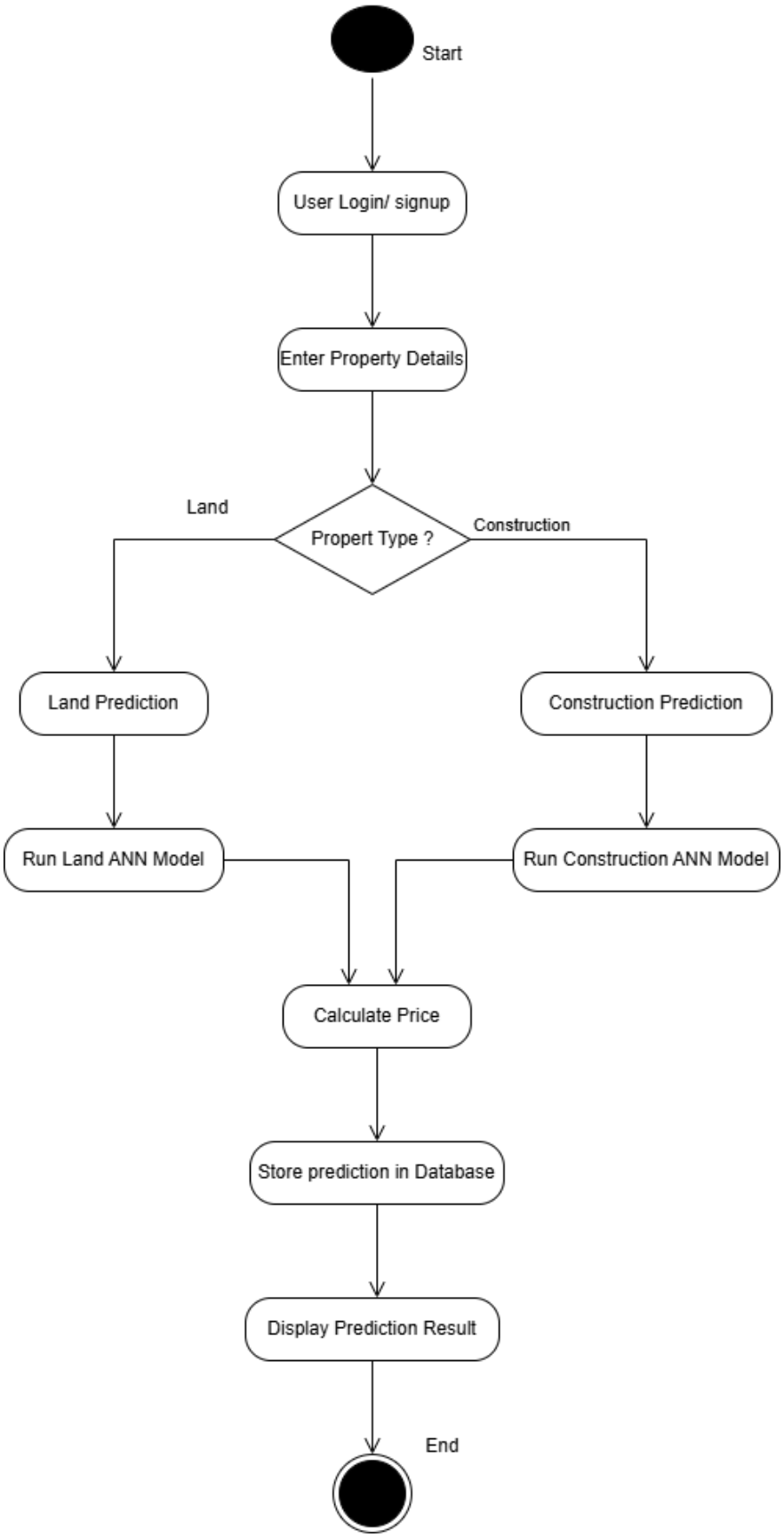


Fig.4.6: Activity Diagram

After either the land prediction or construction prediction model runs, both paths lead to the Calculate Price step. In this stage, the system calculates the final predicted price based on the output generated by the ANN model. Once the price is calculated, the system proceeds to Store Prediction in Database. In this step, the predicted result along with the user input details is saved in the database. This allows the system to maintain prediction records and provide history tracking for users.

Finally, the system moves to Display Prediction Result, where the predicted land price or construction cost is shown to the user through the web interface. This allows the user to view the estimated value instantly. The process ends with the End node, indicating the completion of the prediction workflow. This activity diagram clearly shows how the system processes user inputs, runs the appropriate ANN model, stores the results, and displays the predicted price to the user.

CHAPTER 5

SYSTEM CODING IMPLEMENTA-TION

5. SYSTEM CODING &IMPLEMENTATION

5.1 SOURCE CODE

```
# FINAL (dashboard + history + predictor integration)

from django.shortcuts import render, redirect

from django.conf import settings

from django.contrib import messages

from django.contrib.auth import login, logout

from django.contrib.auth.models import User

from django.contrib.auth.decorators import login_required

from django.db.models import Avg, Count

from django.db.models.functions import TruncDate

from .models import LandQuery

from .land_predictor import predict_land_price

from .forms import PropertyForm

from .models import PropertyQuery, Realtor

from .predictor_construction_ann import predict_construction_cost

from .models import LoginHistory

from django.contrib.auth.decorators import login_required

import os

import joblib

import json

import traceback

import numpy as np

from django.db.models import Avg, Count

from django.db.models.functions import Coalesce
```

```
# PATHS

BASE_DIR = settings.BASE_DIR

PIPELINE_PATH = os.path.join(BASE_DIR, "model", "pipeline_full.joblib")

CLUSTERER_PATH = os.path.join(BASE_DIR, "model", "clusterer_kmeans.joblib")

METRICS_PATH = os.path.join(BASE_DIR, "model", "metrics.json")

CLUSTER_INFO_PATH = os.path.join(BASE_DIR, "model", "cluster_info.json")

LAND_DATASET_PATH = os.path.join(settings.BASE_DIR, "listings", "data",
"up.csv")

pipeline = None

clusterer = None

model_rmse = None

cluster_info = {}

# LOAD MODEL ARTIFACTS

def load_artifacts():

    global pipeline, clusterer, model_rmse, cluster_info

    # pipeline

    try:

        if pipeline is None and os.path.exists(PIPELINE_PATH):

            pipeline = joblib.load(PIPELINE_PATH)

            print("Loaded pipeline:", PIPELINE_PATH)

    except Exception as e:

        print("Pipeline load failed:", e)

        pipeline = None

    # clusterer

    try:

        if clusterer is None and os.path.exists(CLUSTERER_PATH):
```

```
 clusterer = joblib.load(CLUSTERER_PATH)

print("Loaded clusterer:", CLUSTERER_PATH)

    except Exception as e:

        print("Clusterer load failed:", e)

        clusterer = None

    # metrics

    try:

        if os.path.exists(METRICS_PATH):

            with open(METRICS_PATH, "r", encoding="utf-8") as f:

                m = json.load(f)

                # prefer rmse in INR if present, else log rmse

                model_rmse = m.get("rmse_inr") or m.get("rmse_log")

                # ensure numeric or None

                try:

                    model_rmse = float(model_rmse) if model_rmse is not None else None

                except:

                    model_rmse = None

    except Exception as e:

        print("Metrics load failed:", e)

        model_rmse = None

    # cluster_info

    try:

        if os.path.exists(CLUSTER_INFO_PATH):

            with open(CLUSTER_INFO_PATH, "r", encoding="utf-8") as f:

                cluster_info = json.load(f)
```

```
    except Exception as e:
        print("Cluster info load failed:", e)
cluster_info = {}
# initial load
load_artifacts()
# HELPERS
def _to_float(x, default=0.0):
    try:
        if x is None or (isinstance(x, str) and x.strip() == ""):
            return float(default)
        return float(x)
    except:
        return float(default)
def _to_int(x, default=0):
    try:
        if x is None or (isinstance(x, str) and x.strip() == ""):
            return int(default)
        return int(float(x))
    except:
        return int(default)
# AUTH: signup / activation
def signup(request):
    if request.method == "POST":
        email = request.POST.get("email", "").strip().lower()
        password = request.POST.get("password", "")
```

```
if not email or not password:
            return render(request, "registration/signup.html", {
            "error": "Email and password required"
  })
      if User.objects.filter(username=email).exists():
         return render(request, "registration/signup.html", {
            "error": "Email already exists"
         })
      user = User.objects.create_user(
         username=email,
         email=email,
         password=password
      )
      return redirect("login")
return render(request, "registration/signup.html")
```

 # Choose Page

```
@login_required
def choose(request):
   return render(request, "listings/choose.html")
```

 # land home

```
@login_required
def land_home(request):
   return render(request, "listings/land.html")
```

#construction view

```
@login_required
```

```
def index(request):
    result = None
    contacts = []
if request.method == "POST":
        data = {
            "state": request.POST.get("state"),
            "city": request.POST.get("district"),
            "size_sqft": request.POST.get("size_sqft"),
            "bhk": request.POST.get("bedrooms"),
            "age": request.POST.get("age"),
        }
        result = predict_construction_cost(data)
        print("PREDICTED PRICE:", result)
        state = (data.get("state") or "").strip()
        district = (data.get("city") or "").strip()
        if result is None:
            PropertyQuery.objects.create(
            user=request.user,
            state=state,
            district=district,
            location=district,
            size_sqft=float(data.get("size_sqft") or 0),
            bedrooms=int(data.get("bhk") or 0),
            bathrooms=int(request.POST.get("bathrooms") or 0),
            age=int(data.get("age") or 0),
```

```
    predicted_price=float(result)
            )
      print("Construction history saved")
  if state and district:
        contacts = Realtor.objects.filter(
            state__iexact=state,
            district__iexact=district
        )[:6]
        print("STATE:", state)
        print("DISTRICT:", district)
        print("CONTACT COUNT:", contacts.count())
  return render(request, "listings/index.html", {
      "result": result,
      "contacts": contacts
  })
```

#profile

```
@login_required
def profile(request):
  return render(request, "listings/profile.html")
```

#land view

```
@login_required
def land_prediction(request):
  result = None
  contacts = []
  if request.method == "POST":
```

```
    state = (request.POST.get("state") or "").strip()

    city = (request.POST.get("city") or "").strip()

    plot_type = request.POST.get("plot_type")

    area = request.POST.get("Area")

    data = {

        "State": state,

        "District": city,

        "Plot_Type": plot_type

    }

    price_per_sqft = predict_land_price(data)

    print("PRICE PER SQFT:", price_per_sqft)

    area = float(area or 0)

    if price_per_sqft is not None:

        result = float(price_per_sqft) * area

        print("FINAL RESULT:", result)

    else:

        result = 0

    if state and city:

        contacts = Realtor.objects.filter(

            state__iexact=state,

            district__iexact=city

        )[:6]

        try:

            LandQuery.objects.create(

                user=request.user,
```

```
                state=state,
                district=city,
                land_area=area,
                soil_type=plot_type or "Unknown",
                road_width=0.0,
            land_shape="Residential",
                predicted_price=float(result)
            )
        except Exception as e:
            print("Land history save failed:", e)
    return render(request, "listings/land.html", {
        "result": result,
        "contacts": contacts
    })
```

 # HISTORY

```
def history(request):
    qs = PropertyQuery.objects.all().order_by("-created_at")
    records = []
    for r in qs:
        try:
            lakhs = (float(r.predicted_price) / 100000.0) if r.predicted_price is not None else None
        except Exception:
            lakhs = None
        records.append({
            "location": r.location or r.district or r.state,
```

```
        "size_sqft": r.size_sqft,
        "bedrooms": r.bedrooms,
        "bathrooms": r.bathrooms,
        "predicted_price_inr": r.predicted_price,
        "predicted_price_lakhs": lakhs,
        "created_at": r.created_at
})
    return render(request, "listings/history.html", {"records": records})
```

#Land history

```
@login_required
def land_history(request):
    records = LandQuery.objects.all().order_by("-created_at")
    return render(request, "listings/land_history.html", {
        "records": records
    })
```

 # DASHBOARD

```
@login_required
def dashboard(request):
    load_artifacts()
    qs = PropertyQuery.objects.all()
    total = qs.count()
    try:
        avg_price = qs.aggregate(avg=Coalesce(Avg("predicted_price"), 0.0))["avg"] or 0.0
    except Exception:
        avg_price = 0.0
```

```
avg_by_location = []

    try:

        loc_qs = qs.values("location").annotate(avg_price=Coalesce(Avg("predicted_price"), 0.0)).order_by("-avg_price")

        avg_by_location = [(row["location"] or "Unknown", float(round(row["avg_price"], 2))) for row in loc_qs]

    except Exception as e:

        print("avg_by_location compute error:", e)

avg_by_location = []

    cluster_stats = []

    try:

        cluster_qs = qs.exclude(cluster__isnull=True).values("cluster").annotate(cnt=Count("id"), avg_price=Coalesce(Avg("predicted_price"), 0.0)).order_by("cluster")

        for row in cluster_qs:

            cluster_stats.append({

                "cluster": int(row["cluster"]),

                "cnt": int(row["cnt"]),

                "avg_price": float(round(row["avg_price"], 2))

            })

    except Exception as e:

        print("cluster_stats compute error:", e)

        cluster_stats = []

    return render(request, "listings/dashboard.html", {

        "total": total,

        "avg_price": float(round(avg_price, 2)),

        "avg_by_location": avg_by_location,
```

```
        "cluster_stats": cluster_stats,
        "cluster_pca_img": "listings/img/cluster_pca.png",
        "cluster_info": cluster_info
    })
```

#Land dashboard

```
@login_required
def land_dashboard(request):
qs = LandQuery.objects.all()
```

 # STATE-WISE AVERAGE PRICE

```
    state_qs = (
        qs.values("state")
        .annotate(avg_price=Avg("predicted_price"))
        .order_by("state")
    )
    states = [x["state"] for x in state_qs]
    avg_prices = [round(x["avg_price"], 2) for x in state_qs]
    plot_qs = (
        qs.values("soil_type")
        .annotate(cnt=Count("id"))
        .order_by("soil_type")
    )
    plot_labels = [x["soil_type"] for x in plot_qs]
    plot_counts = [x["cnt"] for x in plot_qs]
```

```
    # PREDICTION TREND (DATE-WISE)

    trend_qs = (

        qs.annotate(day=TruncDate("created_at"))

        .values("day")

        .annotate(avg_price=Avg("predicted_price"))

        .order_by("day")

    )

    trend_dates = [x["day"].strftime("%Y-%m-%d") for x in trend_qs]

    trend_prices = [round(x["avg_price"], 2) for x in trend_qs]

    context = {

        "states": json.dumps(states),

"avg_prices": json.dumps(avg_prices),

        "plot_labels": json.dumps(plot_labels),

        "plot_counts": json.dumps(plot_counts),

        "trend_dates": json.dumps(trend_dates),

        "trend_prices": json.dumps(trend_prices),

    }

    return render(request, "listings/land_dashboard.html",context)

# STATIC PAGES

def about(request):

    return render(request, "listings/about.html")

def details(request):

    context = {

        "algorithm": "Regression pipeline",

        "framework": "Django + scikit-learn",
```

```
 "features": ["state", "city", "locality", "size_sqft", "bedrooms", "bathrooms", "age"],
        "output": "Predicted price (₹)" }
    return render(request, "listings/details.html", context)
@login_required
def land_details(request):
    context = {
        "algorithm": "Artificial Neural Network (ANN)",
        "model_type": "Deep Learning Regression",
        "features": [
            "State",
            "City",
            "Plot Type",
            "Land Area (sq.ft)"
        ],
        "output": "Predicted Land Price (₹)",
        "dataset": "Historical land price dataset (up.csv)"
    }
    return render(request, "listings/land_details.html", context)
```

#logout

```
def logout_view(request):
    logout(request)
    return redirect("login")
def landing(request):
    if request.user.is_authenticated:
        return redirect('choose')
```

```
    return render(request, "listings/landing.html")
```

#Login history

```
@login_required
def login_history(request):
    records = LoginHistory.objects.filter(user=request.user).order_by('-login_time')
    return render(request, "listings/login_history.html", {
        "records": records
    })
```

5.2 SOFTWARE TESTING

Software testing is the process of evaluating and verifying that a software application or system functions as intended and meets specified requirements. It aims to identify defects, errors, and gaps to ensure the software's quality, reliability, and performance. The primary objectives of software testing include detecting errors, validating and verifying system functionality, ensuring performance efficiency, and checking for security vulnerabilities. Testing can be performed manually, where testers execute test cases without automation, or automatically, using scripts and tools to speed up the process. By conducting thorough testing, organizations can improve software quality, enhance user satisfaction, and minimize potential risks before deployment.

5.2.1 TESTING METHODOLOGIES

The following are the Testing Methodologies:

- Unit Testing
- Integration Testing
- User Acceptance Testing
- Output Testing
- Validation Testing

Unit Testing

Unit testing is performed to verify the functionality of individual components of the system separately. In this project, different modules such as the land price prediction function, construction cost prediction function, dataset preprocessing functions, and model loading functions are tested independently. Each function is provided with sample inputs to ensure that it produces the expected output without errors. For example, the prediction functions are tested by giving different values for area, BHK, bathrooms, and property age to verify whether the ANN model returns reasonable price predictions. This testing helps identify errors at an early stage and ensures that every module works correctly before integrating it with the complete system.

Integration Testing

Integration testing checks whether different modules of the system work together smoothly after combining them. In this project, several components such as the Django web interface, backend server logic, trained ANN models, preprocessing module, and database are integrated and tested together. When a user submits property details through the web form, the data should be sent to the backend, processed correctly, passed to the ANN model for prediction, and the predicted result should be displayed on the web page. Integration testing ensures that all these modules communicate properly without errors and that the system works as a complete application.

User Acceptance Testing

User acceptance testing is conducted to ensure that the system satisfies the needs and expectations of the end users. In this project, users interact with the system by registering an account, logging into the system, entering property details, and viewing predicted prices**.** The system is tested from the user's perspective to ensure that the interface is user-friendly and the predictions are understandable. Feedback from users helps verify whether the system is easy to operate and whether it provides useful information for real estate price estimation and decision making.

Output Testing

Output testing is performed to verify whether the system produces correct and meaningful results after processing the input data. In this project, the predicted land prices and construction costs generated by the ANN models are checked to ensure that they fall within reasonable ranges. The system is tested with different input combinations to confirm that the predicted values are displayed correctly on the web interface. Additionally, the output is verified to ensure that it is properly stored in the database and appears correctly in the prediction history and dashboard sections.

Validation Testing

Validation testing ensures that the system accepts only valid and meaningful input values from users. In this project, validation checks are applied to ensure that required fields such as state, district, property size, BHK, bathrooms, and property age are entered correctly before processing the prediction request. The system prevents invalid inputs such as empty fields, negative values, or incorrect formats. By validating user inputs before sending them to the ANN model, the system reduces errors and ensures accurate and reliable predictions.

5.2.2 TESTING STRATAGIES

A strategy for system testing integrates system test cases and design techniques into a well planned series of steps that results in the successful construction of software. The testing strategy must co-operate test planning, test case design, test execution, and the resultant data collection and evaluation. A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high level tests that validate major system functions against user requirements.

Software testing is a critical element of software quality assurance and represents the ultimate review of specification design and coding. Testing represents an interesting anomaly for the software. Thus, a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

System Testing

System testing is performed to evaluate the complete and integrated system to ensure that all components work together correctly. In this project, the entire Real Estate Price Prediction System is tested after integrating the frontend interface, backend logic, ANN prediction models, and database. The testing verifies whether users can successfully register, log in, enter property details, and obtain predicted land prices or construction costs without errors.. System testing ensures that the application functions smoothly, meets the project requirements, and provides accurate and reliable results to users.

Unit Testing

Unit testing is used to verify that individual components of the system work correctly. In this project, each module such as the land price prediction function, construction cost prediction function, data preprocessing module, and model loading module is tested separately. Sample inputs like state, district, plot type, area, BHK, bathrooms, and property age are given to the functions to check whether they produce the expected results. This testing ensures that each part of the system performs its task correctly before combining it with other modules

5.2.3 TYPES OF TESTS

Unit Testing

Unit testing is performed to verify the functionality of individual components of the system independently. In this project, modules such as the land price prediction function, construction cost prediction function, preprocessing functions, and model loading functions are tested separately. Sample inputs like state, district, plot type, area, BHK, bathrooms, and property age are used to ensure that each function produces the correct output.

Integration Testing

Integration testing checks whether different modules of the system work together correctly. In this project, the Django web interface, backend logic, ANN prediction models, and database are integrated and tested together. This ensures that the user input is properly processed, sent to the prediction model, and the predicted result is displayed and stored in the database.

System Testing

System testing evaluates the complete system to ensure that it functions according to the project requirements. In this project, the entire application including user authentication, property input forms, ANN prediction models, prediction history, and agent contact modules is tested as a whole. This testing ensures that the full system works smoothly without errors.

Functional Testing

Functional testing verifies whether the system performs all the required functions correctly. In this project, functions such as user registration, login, land price prediction, construction cost prediction, viewing prediction history, and displaying agent contact details are tested to confirm that each feature works as expected.

User Acceptance Testing

User acceptance testing is performed to ensure that the system meets the needs of end users. In this project, users test the system by registering, logging in, entering property details, and viewing predicted prices. This testing confirms that the system is easy to use and provides useful real estate price predictions for users.

CHAPTER 6
EXECUTION STEPS

6. EXECUTION STEPS

6.1 INSTALLATION OF SOFTWARE

We need to set up the required software environment on the system to run the Real Estate Price Prediction System. This project is developed using Python, Django, and TensorFlow, so Python must be installed on the system. Python provides the base platform for running the application, while Django is used for web development and TensorFlow is used for building and running the ANN models. Therefore, Python 3.x or later must be installed before proceeding with the project setup.

Step 1: Verify Python Installation

First, check whether Python is already installed on the system or not. Open the command prompt and type the following command:

python --version

If Python is already installed, it will display the version number. In my case Python is not installed on the system.

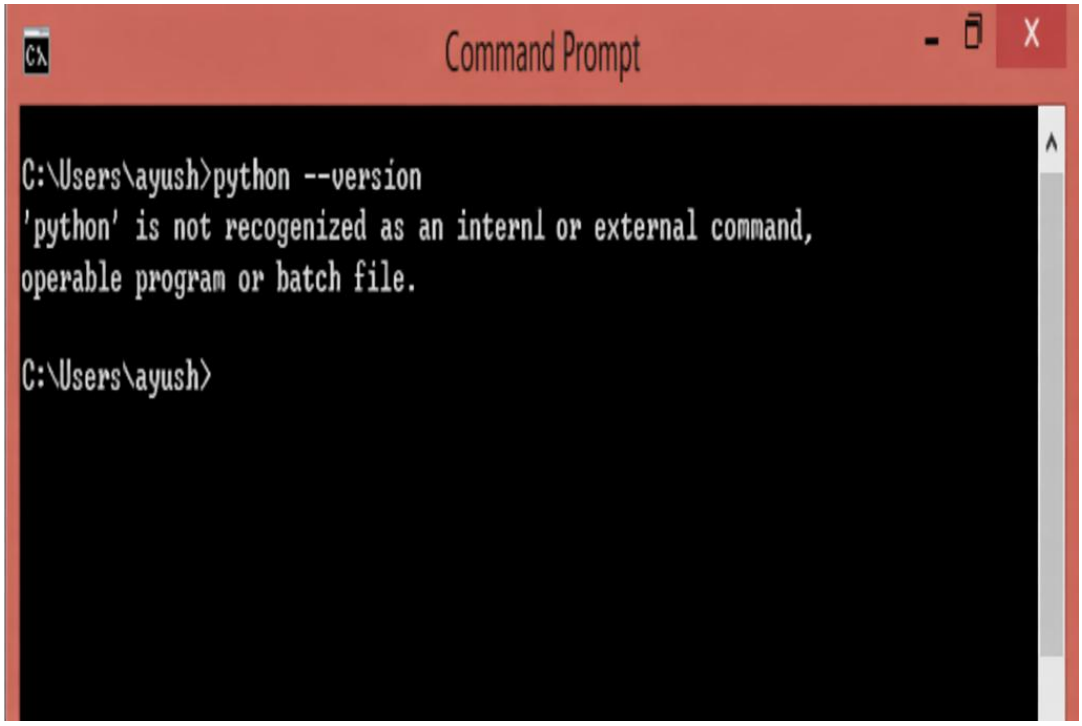


Fig.6.1: Python Verification

Step 2: Download Python

Visit the official Python website **https://www.python.org/downloads/** and download the latest version of Python suitable for your operating system.

Fig.6.2: Download Python

Step 3: Run Python Installer

After downloading, run the installer file. In this window you must check the option "Add Python to PATH" before starting installation. This allows Python to run from the command prompt.

Fig.6.3: Python Installing

Step 4: Python Installation Process

Click Install Now to begin installation. The system installs Python and required libraries automatically. The progress bar shows the installation status.

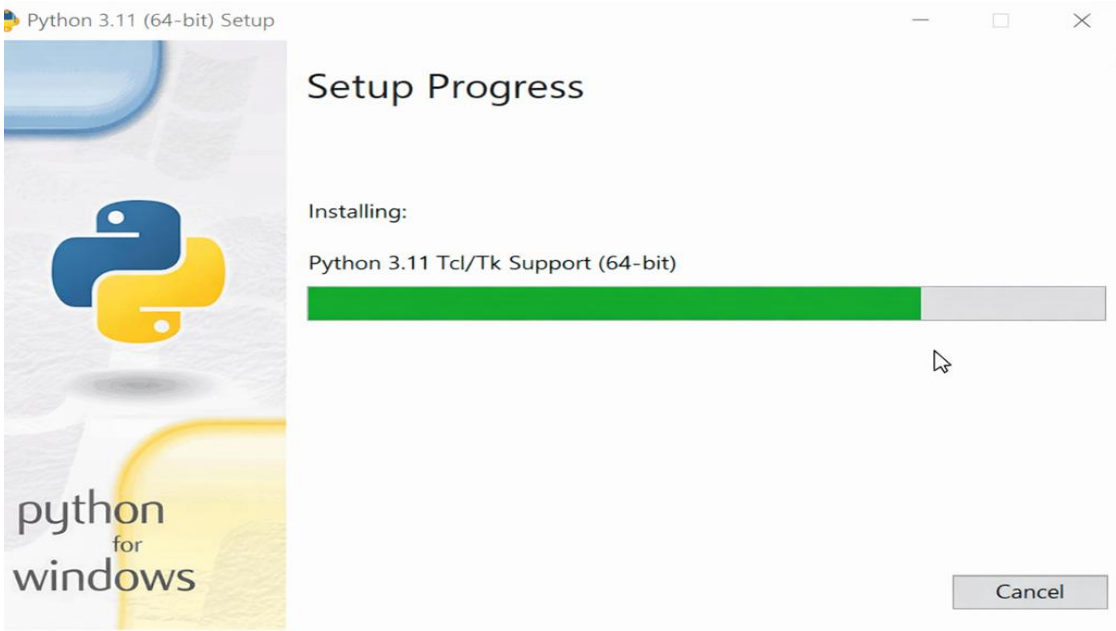


Fig.6.4: Python Installation Process

Step 5: Installation Successful

Fig .6.5: Installation Successful

6.2 INSTALLATION OF DJANGO

Django is a high-level Python web framework used to develop the web-based interface of the Real Estate Price Prediction System. It helps in handling user requests, managing databases, and displaying prediction results. Before installing Django, Python must be installed on the system. Django is installed using the pip package manager.

Step 1: Open the Command Prompt and install Django using the following command:

pip install django

This command downloads and installs the Django framework along with the required dependencies.

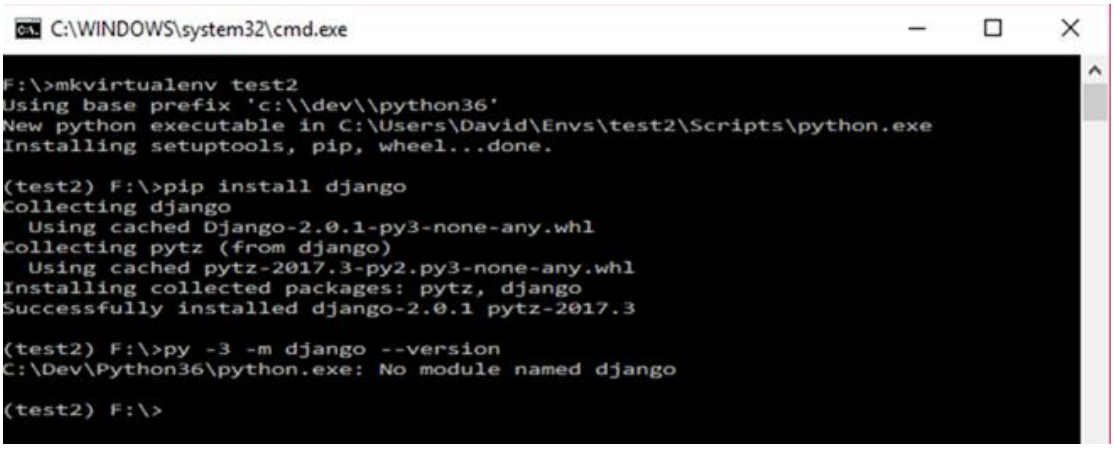


Fig.6.6: Django Installation

Step 2: Verify Django Installation

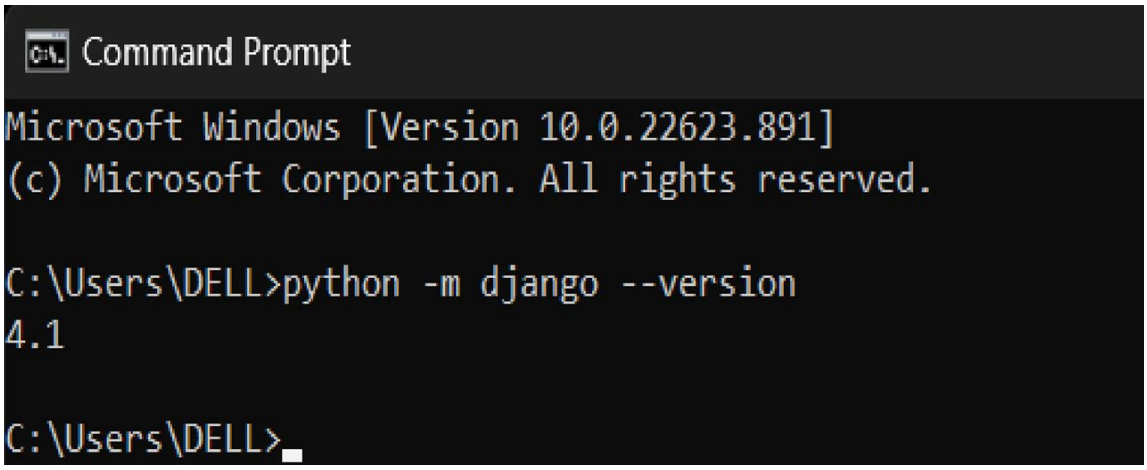


Fig.6.7: Django Version

After installation, verify whether Django is installed successfully by running the following command:

Python -m django--version

If Django is installed correctly, the version number will be displayed on the screen.

Step 3: Run Django Server

This starts the Django development server on the local machine.

```
(djangoWebsite3.9) → django-website python manage.py runserver 0.0.0.0:8000
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
December 15, 2020 - 06:49:59
Django version 3.1.4, using settings 'hackershack_website.settings'
Starting development server at http://0.0.0.0:8000/
Quit the server with CONTROL-C.
[15/Dec/2020 06:50:06] "GET / HTTP/1.1" 200 526
[15/Dec/2020 06:50:09] "GET /about HTTP/1.1" 200 510
[15/Dec/2020 06:50:10] "GET /contact HTTP/1.1" 200 512
[15/Dec/2020 06:50:11] "GET /accounts/profile HTTP/1.1" 200 840
[15/Dec/2020 06:50:14] "GET / HTTP/1.1" 200 526
^C
```

Fig.6.8: Running Django Server

Step 4:. Django default page displayed in browser.

If the installation is successful, the default Django welcome page will be displayed, indicating that the framework is working correctly.

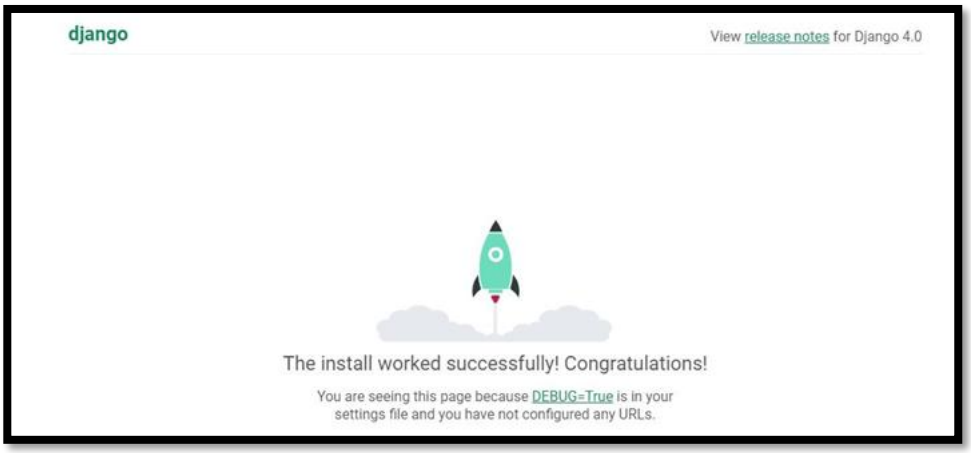


Fig.6.9:Django Welcome Page

6.3 Required Libraries

Libraries play an important role in the development of project by supporting data processing and model building. In this system, libraries such as TensorFlow, Pandas, NumPy, and Joblib are used to handle machine learning tasks, data manipulation, and model storage. These libraries help in processing the dataset, training the ANN models, and generating accurate predictions. Together, they ensure efficient performance and reliable results in the Real Estate Price Prediction System.

NumPy

NumPy is used for numerical computations and array operations. It helps in efficient mathematical calculations during model training and prediction. This is command to install numpy:

Pip install numpy

Pandas

Pandas is used for handling and processing the dataset. It helps in reading CSV files and organizing data into structured format. This is command to install pandas:

Pip install pandas

Joblib

Joblib is used to save and load trained models and preprocessing objects. It improves performance by storing models for future use. This is command to install joblib:

Pip install joblib

TensorFlow

TensorFlow is used to build and train the Artificial Neural Network (ANN) models. It helps in predicting land prices and construction costs based on input data. This is command to install Tensorflow:

Pip install tensorflow

CHAPTER 7
SCREENSHOTS

7. SCREENSHOTS

This is the execution of the Django project using the command python manage.py runserver in the VS Code terminal. The system successfully initializes and loads both the Land ANN model and Construction ANN model, indicating that the machine learning models are ready for prediction. The terminal displays some TensorFlow-related messages, which indicate that the system is optimized for CPU performance. A warning related to static files directory is also shown, which does not affect the execution of the project.

Finally, the message Starting development server at "**http://127.0.0.1:8000/**" confirms that the Django server is running successfully, and the web application can now be accessed through a browser.

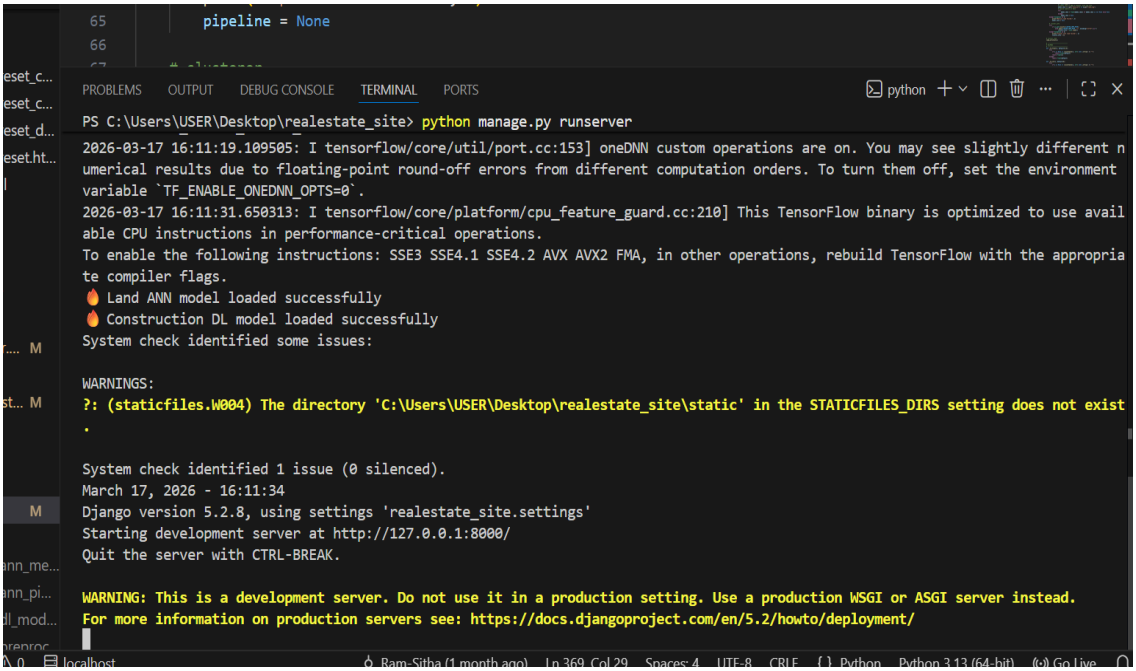


Fig.7.1:Running the Project

Welcome page of the Real Estate Price Prediction System, which acts as the entry point for users. The page provides options for users to either log in or create a new account to access the system features. The interface displays a welcome message and a brief description of the system, highlighting its ability to provide instant property price predictions based on state and city data.

The presence of "Log in" and "Sign up" buttons enables secure user authentication before accessing prediction functionalities. Additionally, the page showcases key features such as instant predictions, clean reports, and location-based analysis, making the system user-friendly and informative.

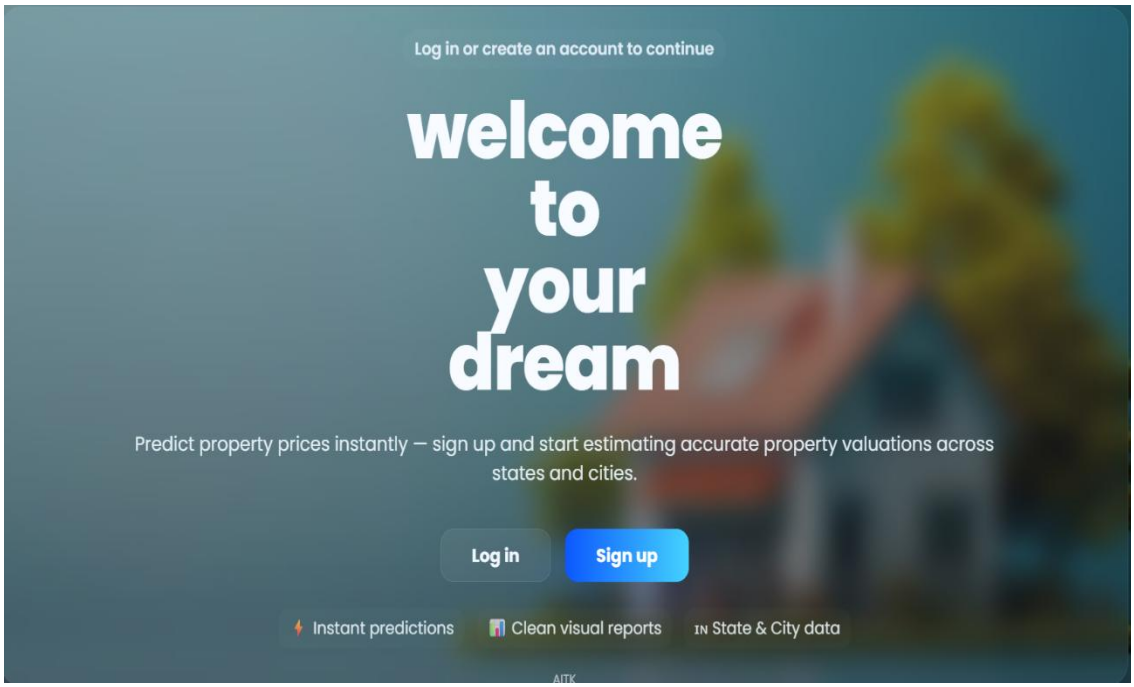


Fig.7.2: Welcome Page

Signup page shows the user registration page of the Real Estate Price Prediction System. It allows new users to create an account to access the prediction features of the system. The form collects essential details such as email address, password, and confirm password, ensuring secure account creation. Password guidelines are provided to help users create strong credentials, improving system security.

Once the user enters the required details and clicks on the "Create Account" button, the information is stored securely in the database. The page also provides a link for existing users to log in, making the system easy to navigate.

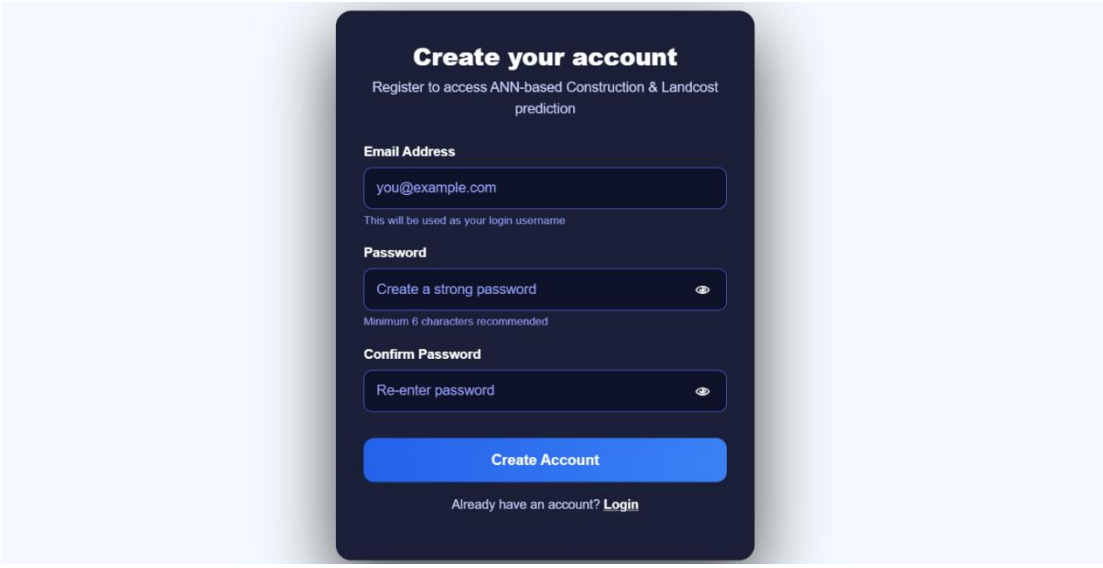


Fig.7.3: User Signup Page

This is login page of the Real Estate Price Prediction System. It allows registered users to securely access the system by entering their email address and password. The interface ensures secure authentication by verifying the entered credentials with the stored user data in the database.

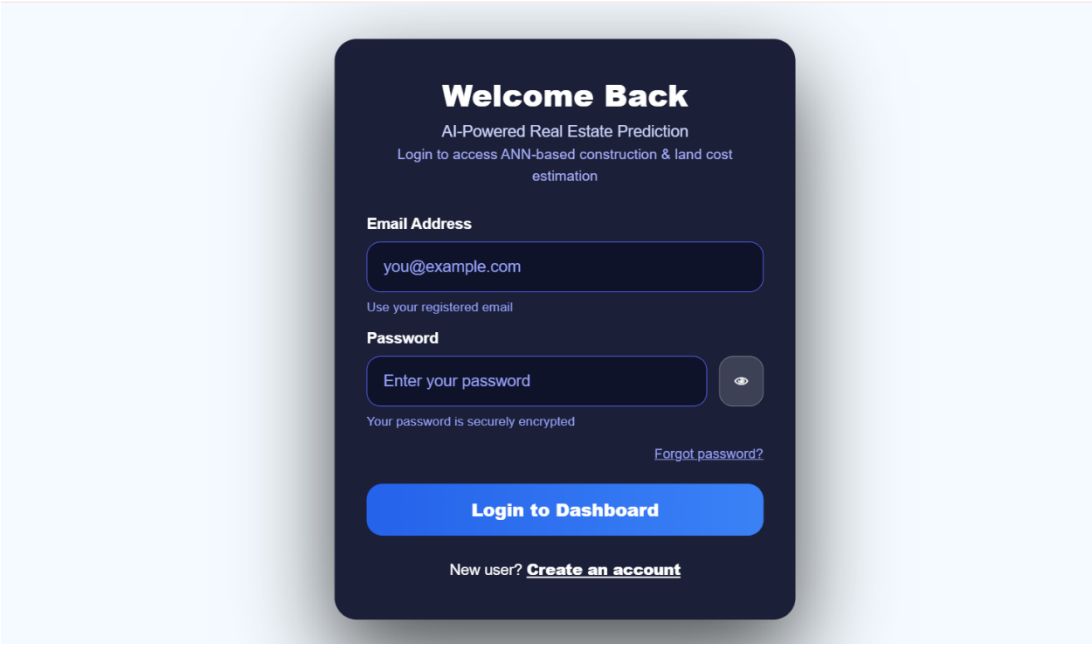


Fig.7.4: User Login Page

It also provides additional options such as “Forgot password” for account recovery and a link for new users to create an account. Once the user clicks on “Login to Dashboard”, they are redirected to the main application where they can perform land price and construction cost predictions.

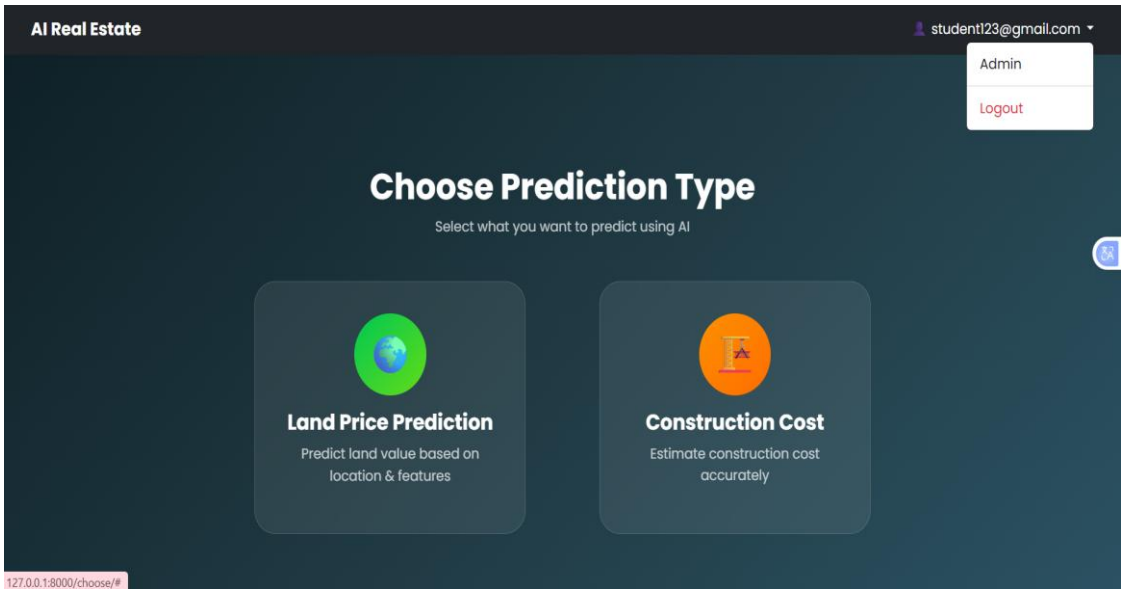


Fig.7.5: Home Page

Home page shows prediction type selection page of the Real Estate Price Prediction System. After successful login, users are presented with options to choose the type of prediction they want to perform. The system provides two main options: Land Price Prediction and Construction Cost Prediction. Users can select either option based on their requirement. It also displays user account details and logout functionality for better user management. At the top right corner, a user dropdown menu is available, which displays options such as Admin/Profile and Logout. This allows users to manage their account and securely exit the system.

Land Prediction Pages

Input page shows the Land Price Prediction module interface of the system. It allows users to enter required details such as state, city, plot type, and land area (in square feet) for predicting land price. The input fields are designed using dropdowns and text boxes to ensure accurate and user-friendly data entry.

Once the user provides all the required inputs, they can click on the "Price Prediction" button to generate the estimated land price using the trained ANN model. At the top navigation bar, options like Home, History, Dashboard, Details, and Switch are available, allowing users to navigate through different features of the system easily.

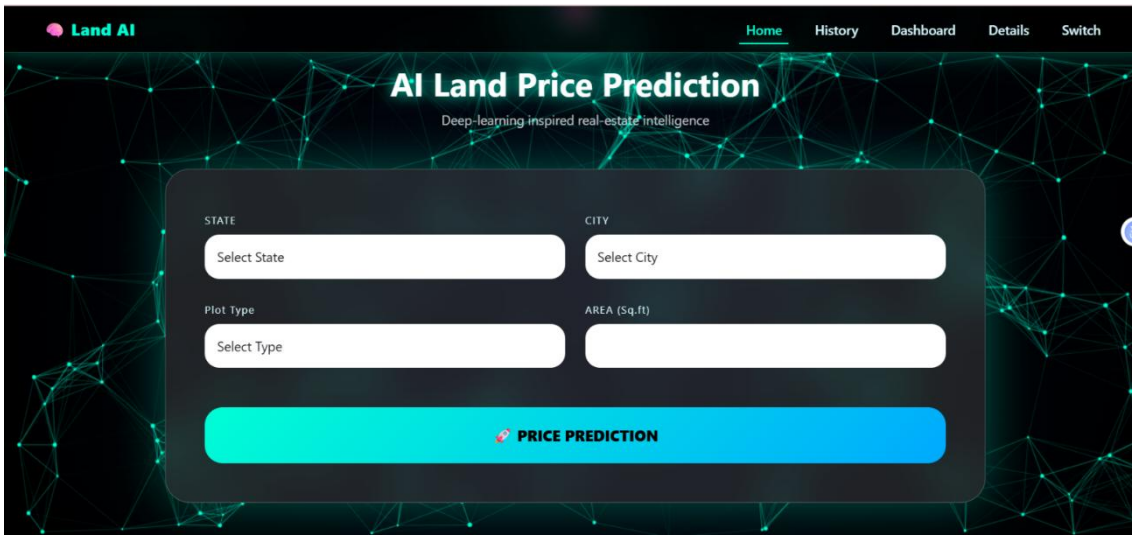


Fig.7.6: Land Prediction Input Page

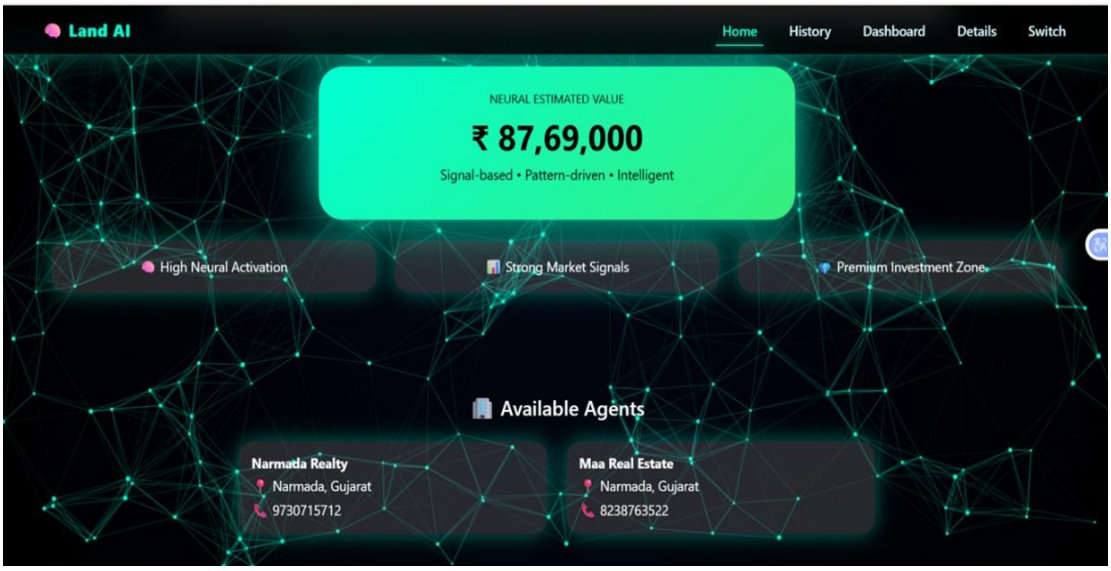
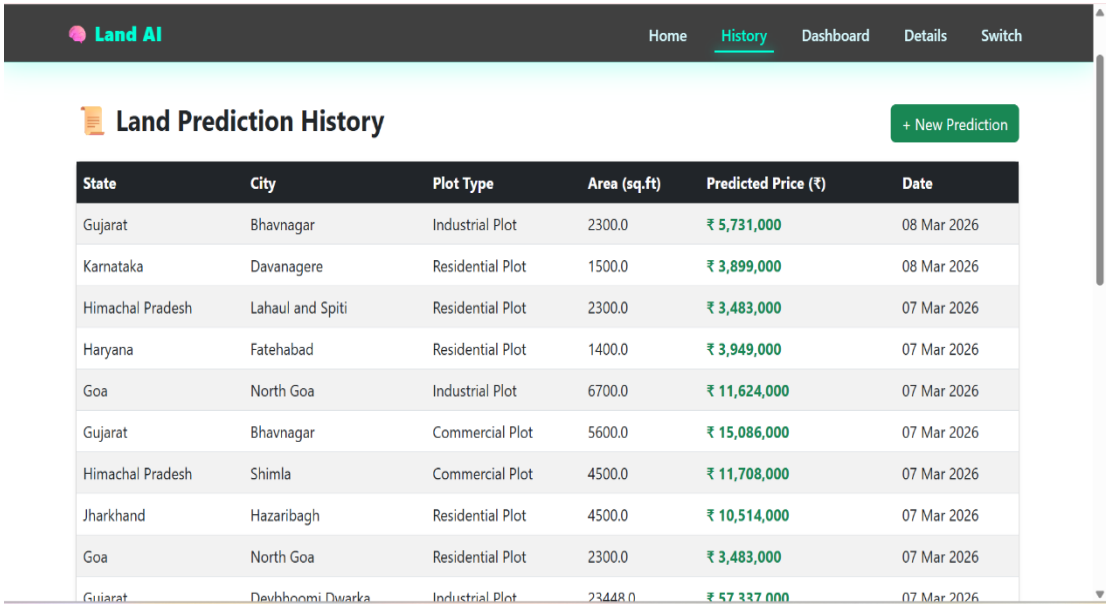


Fig.7.7: Land Prediction Result Page

Result page shows the output page of the Land Price Prediction module. After entering the required inputs, the system processes the data using the trained ANN model

and displays the predicted land price. The predicted value is highlighted as "Neural Estimated Value", indicating that the result is generated using deep learning techniques. Below the price, descriptive insights such as High Neural Activation, Strong Market Signals, and Premium Investment Zone are shown to give users a better understanding of the prediction.

Additionally, the system displays available real estate agents based on the selected location, including their contact details. This helps users connect with professionals for further assistance, making the system more practical and user-friendly.



State	City	Plot Type	Area (sq.ft)	Predicted Price (₹)	Date
Gujarat	Bhavnagar	Industrial Plot	2300.0	₹ 5,731,000	08 Mar 2026
Karnataka	Davanagere	Residential Plot	1500.0	₹ 3,899,000	08 Mar 2026
Himachal Pradesh	Lahaul and Spiti	Residential Plot	2300.0	₹ 3,483,000	07 Mar 2026
Haryana	Fatehabad	Residential Plot	1400.0	₹ 3,949,000	07 Mar 2026
Goa	North Goa	Industrial Plot	6700.0	₹ 11,624,000	07 Mar 2026
Gujarat	Bhavnagar	Commercial Plot	5600.0	₹ 15,086,000	07 Mar 2026
Himachal Pradesh	Shimla	Commercial Plot	4500.0	₹ 11,708,000	07 Mar 2026
Jharkhand	Hazaribagh	Residential Plot	4500.0	₹ 10,514,000	07 Mar 2026
Goa	North Goa	Residential Plot	2300.0	₹ 3,483,000	07 Mar 2026
Gujarat	Devbhoomi Dwarka	Industrial Plot	23448.0	₹ 57,337,000	07 Mar 2026

Fig.7.8: Land Prediction History page

History page shows the Land Prediction History page of the system. It displays all the previous land price predictions made by the user in a tabular format. The table includes important details such as state, city, plot type, area (in sq.ft), predicted price, and date, allowing users to easily track and review their past predictions. Each record represents a prediction generated using the ANN model and stored in the database.

A "+ New Prediction" button is also provided, which allows users to quickly navigate back to the prediction module and perform a new estimation. This feature helps in maintaining a record of user activity and improves usability by providing easy access to past data.

Dashboard page of the Land Price Prediction System, which provides analytical insights based on prediction data. It helps users understand trends and patterns in real estate pricing. The dashboard includes a bar chart representing state-wise average prices, allowing users to compare property values across different states. It also displays a pie chart showing plot type distribution (Commercial, Industrial, Residential), giving a clear overview of market composition.

These visualizations are generated from stored prediction data and help users make informed decisions by analyzing market trends and property distribution in an easy and interactive way.

Fig.7.9: Land Prediction Dashboard

Construction Prediction Pages

This is Construction Cost Prediction module interface of the system. It allows users to enter required details such as state, district (city), size (in sq.ft), number of bedrooms (BHK), number of bathrooms, and age of the property. These inputs represent important structural and location-based factors that influence construction cost. The form is designed with user-friendly input fields to ensure accurate data entry.

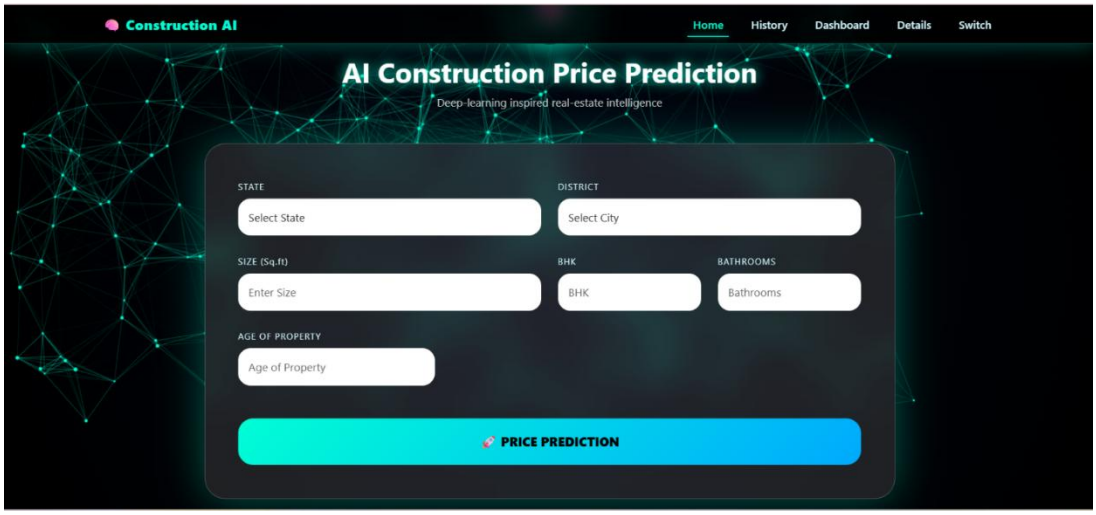


Fig.7.10: Construction Prediction Input Page

After providing all the required details, the user can click on the "Price Prediction" button, which sends the data to the ANN model to estimate the construction cost. The top navigation bar also provides access to other features like Home, History, Dashboard, Details, and Switch.

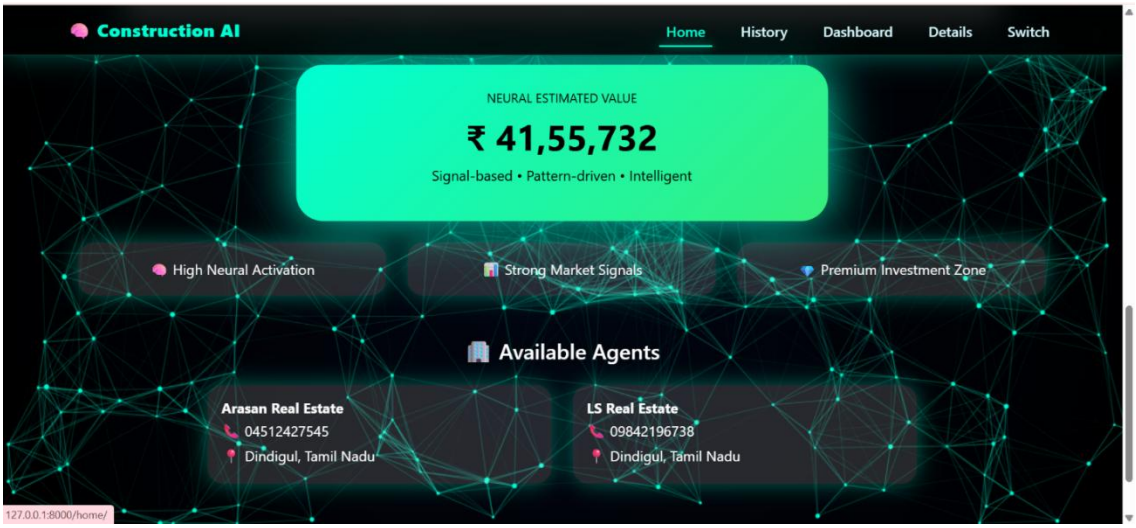
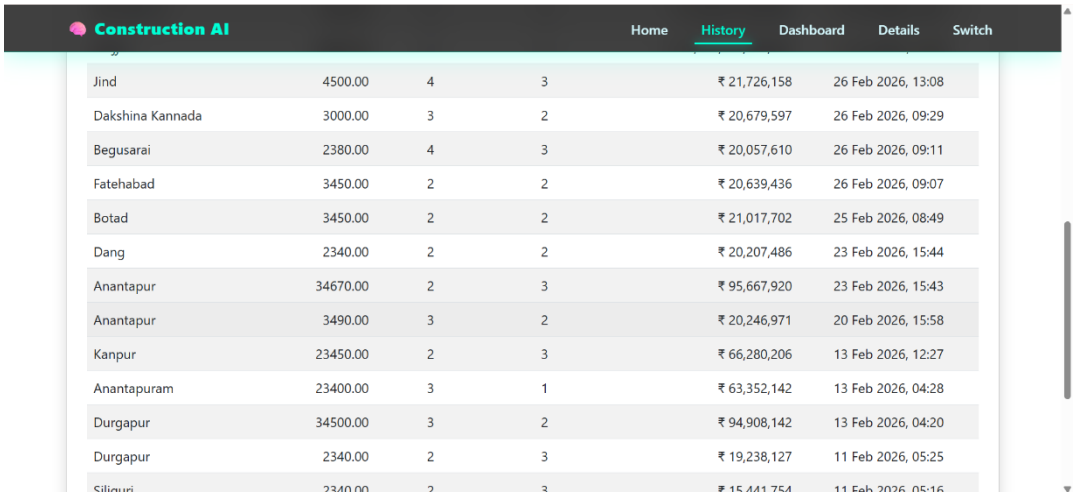


Fig.7.11: Construction Prediction Result Page

Result page shows the output page of the Construction Price Prediction module. After entering the required inputs, the system processes the data using the trained ANN model and displays the predicted land price. The predicted value is highlighted as "Neural

Estimated Value", indicating that the result is generated using deep learning techniques. Below the price, descriptive insights such as High Neural Activation, Strong Market Signals, and Premium Investment Zone are shown to give users a better understanding of the prediction. Additionally, the system displays available real estate agents based on the selected location, including their contact details. This helps users connect with professionals for further assistance, making the system more practical and user-friendly.



Jind	4500.00	4	3	₹ 21,726,158	26 Feb 2026, 13:08
Dakshina Kannada	3000.00	3	2	₹ 20,679,597	26 Feb 2026, 09:29
Begusarai	2380.00	4	3	₹ 20,057,610	26 Feb 2026, 09:11
Fatehabad	3450.00	2	2	₹ 20,639,436	26 Feb 2026, 09:07
Botad	3450.00	2	2	₹ 21,017,702	25 Feb 2026, 08:49
Dang	2340.00	2	2	₹ 20,207,486	23 Feb 2026, 15:44
Anantapur	34670.00	2	3	₹ 95,667,920	23 Feb 2026, 15:43
Anantapur	3490.00	3	2	₹ 20,246,971	20 Feb 2026, 15:58
Kanpur	23450.00	2	3	₹ 66,280,206	13 Feb 2026, 12:27
Anantapuram	23400.00	3	1	₹ 63,352,142	13 Feb 2026, 04:28
Durgapur	34500.00	3	2	₹ 94,908,142	13 Feb 2026, 04:20
Durgapur	2340.00	2	3	₹ 19,238,127	11 Feb 2026, 05:25

Fig.7.12: Construction Prediction History Page

History page shows the Construction Cost Prediction History page of the system. It displays all previously generated construction cost predictions in a structured table format. The table includes details such as location (district), size (sq.ft), number of bedrooms (BHK), number of bathrooms, predicted cost, and date/time, allowing users to track their past predictions. Each entry represents a result generated by the ANN model and stored in the database. This feature helps users review and compare different construction estimates over time, making the system more useful for planning and decision-making.

Dashboard module of the Construction AI system, which provides analytical insights based on previous predictions. It displays important metrics such as total number of predictions and average predicted price, giving an overview of system usage and results. The dashboard includes a bar chart showing average price by location, which helps users understand how construction costs vary across different regions.

It also contains a pie chart representing location-wise distribution, giving a visual representation of prediction frequency for each area.

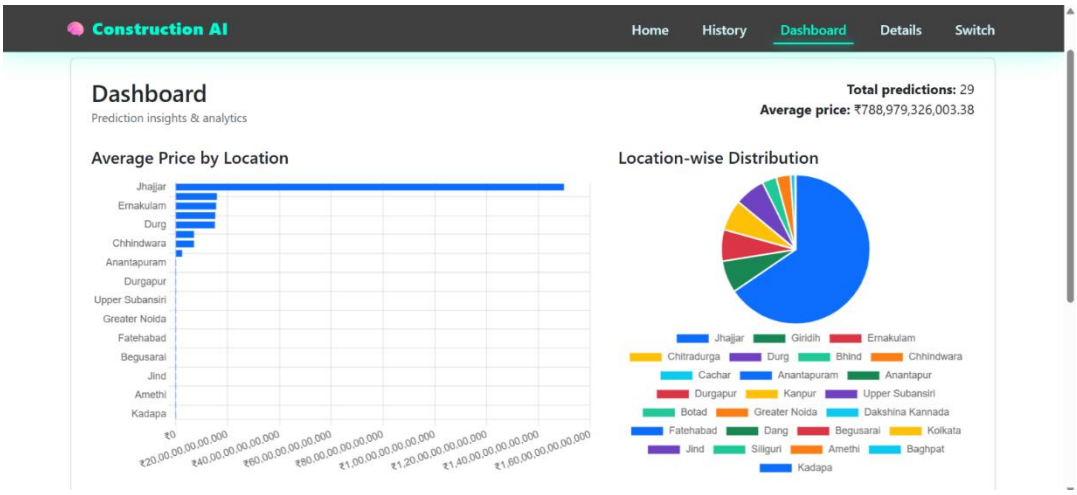


Fig.7.13: Construction Prediction Dashboard

This module helps users analyze trends, compare prices across locations, and make informed decisions based on historical prediction data. It enhances the system by providing both visual analytics and data-driven insights in an easy-to-understand format.

Admin Pages

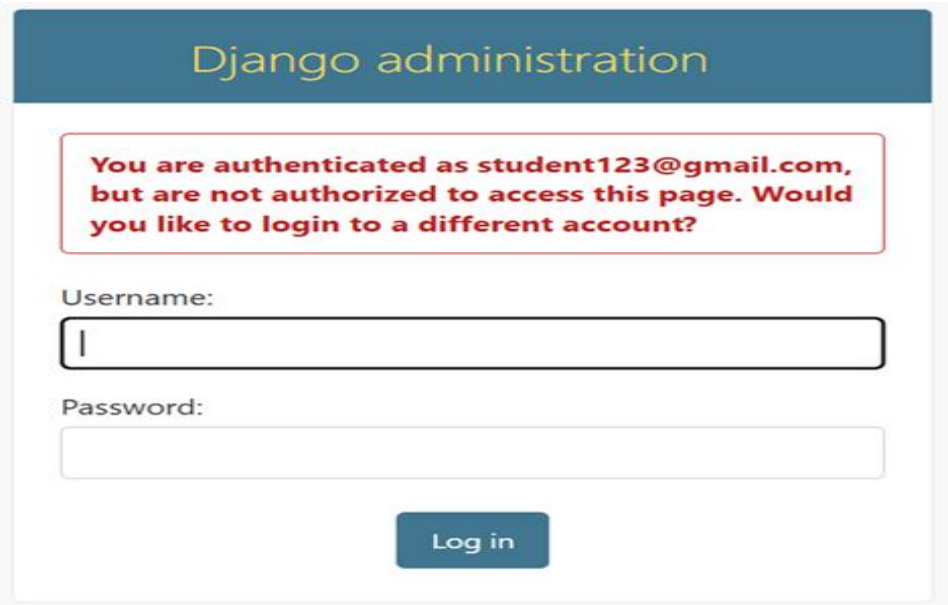


Fig.7.14: Admin Login Page

Admin page can access from the choose page, the system provides a navigation option that redirects the user to the admin login interface. When the user selects the Admin option from the choose page, the request is routed to the admin login URL configured in the Django application. After entering valid admin credentials, the user is successfully logged in and redirected to the Admin Dashboard. From there, the admin can manage user data, prediction history, and agent details efficiently.

If a normal user tries to access the admin panel, the system displays a message stating that the user is not authorized, as shown in the above figure. This ensures security by restricting access only to authorized administrators. The admin must log in using valid superuser credentials to manage system data such as users, predictions, and agent details.

Django Admin Dashboard of the system, After successful login with admin credentials, the admin is redirected to this page. It provides a centralized interface to manage all the data in the application. The admin can manage different modules such as Users, Property Queries, Login History, and Realtors.

Using the “Add” and “Change” options, the admin can insert new data, update existing records, and monitor user activities. The dashboard also displays recent actions, which helps the admin track recent updates and changes made in the system. This ensures better control, data management, and monitoring of the Real Estate Price Prediction System.

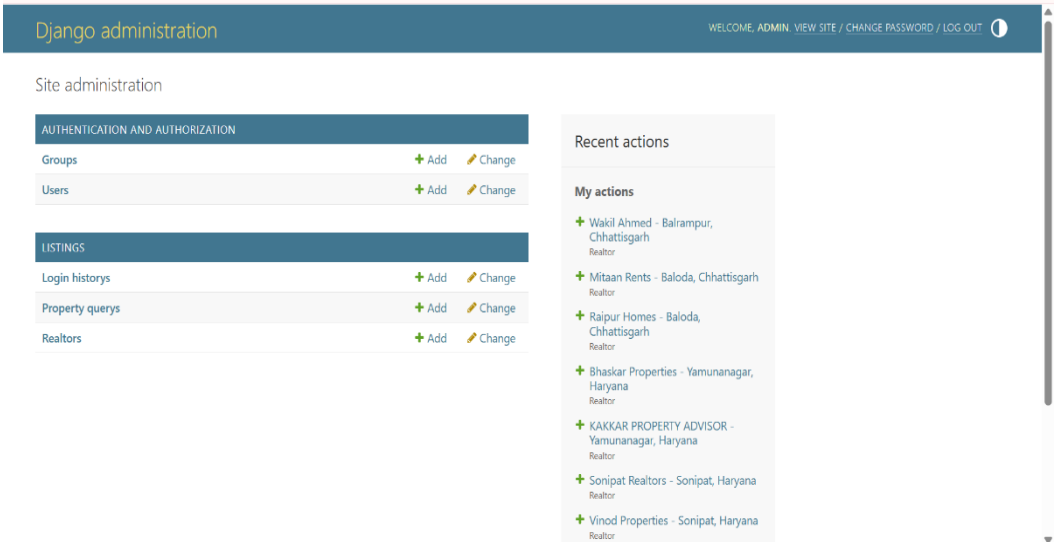


Fig.7.15: Admin Home Page

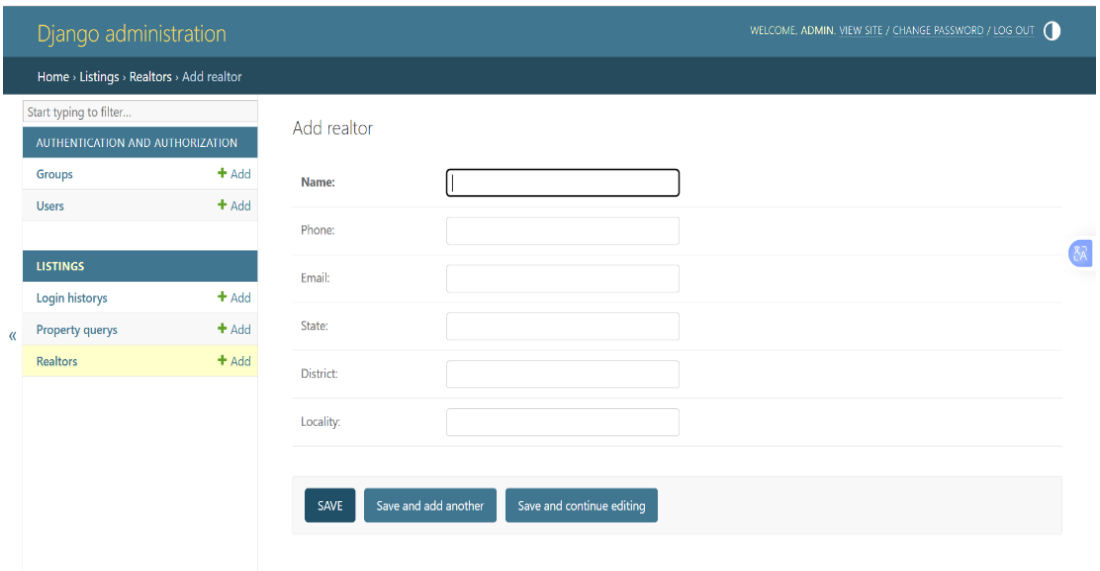


Fig.7.16: Add Realtor Page

Add Realtor page in the Django Admin panel allows the admin to add new real estate agent details into the system.The admin enters important information such as Name, Phone, Email, State, District, and Locality. After filling the required details, the admin can click on the Save button to store the agent information in the database. The options like "Save and add another" and "Save and continue editing" provide flexibility for managing multiple records. This feature helps in maintaining updated and accurate agent contact details, which are later displayed to users in the system.

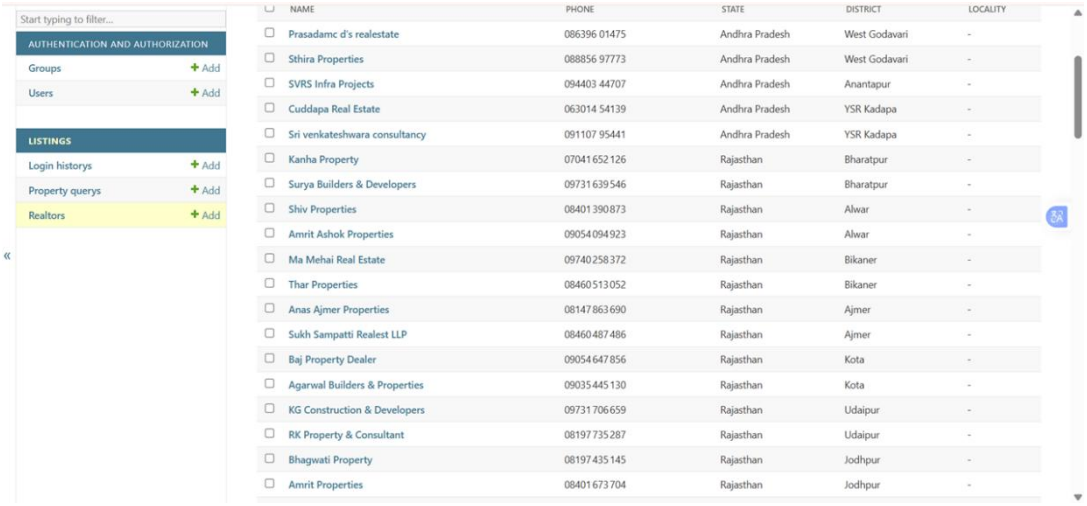


Fig.7.17: Realtors Page

Realtors list page in the Django Admin panel will displays all the real estate agents stored in the system in a tabular format. Each record includes details such as Name, Phone, State, District, and Locality. The admin can easily view, edit, or delete any realtor information using this interface. This page helps the admin to manage and monitor all agent details efficiently. The stored realtor data is later used to display relevant agent contacts to users based on their selected location.

CHAPTER 8
CONCLUSION

8. CONCLUSION

The project presents a Real Estate Price Prediction System developed using Artificial Neural Networks (ANN) to provide accurate estimation of land and construction costs. Real estate pricing depends on various factors such as location, area, and property type, making manual estimation difficult and unreliable. To address this, the system uses ANN models trained on historical data and considers features like state, district, plot type, land area, built-up area, and number of rooms. It is implemented as a web application using Django, allowing users to enter property details and receive instant predictions. The backend processes data through encoding and scaling before generating results. The system also stores user queries, provides admin management features, and includes agent contact details to assist users. Overall, it offers a fast, accurate, and data-driven solution for better real estate decision-making.

CHAPTER 9
FUTURE ENHANCEMENTS

9. FUTURE ENHANCEMENTS

The future enhancements of this project focus on improving accuracy, usability, and system performance. Integration of real-time real estate data using APIs will help the model stay updated with market trends and provide more reliable predictions. The system can be improved by adding location-based prediction using GPS or maps for precise price estimation. This will help analyze variations within the same locality. Developing a mobile application for Android and iOS will increase accessibility and improve user experience. Advanced deep learning models can be used to enhance prediction performance. Features like data visualization and price trend analysis can also be included. Recommendation systems can guide users in making better investment decisions. Integration with government land records will ensure more authentic data. Overall, these enhancements will make the system more accurate, scalable, and user-friendly.

CHAPTER 10
BIBLIOGRAPHY

10. BIBLIOGRAPHY

- EEE Xplore. (2025). "Deep-Learning Approach for an Analysis of Real-Estate Prices and Transactions Available at: https://ieeexplore.ieee.org/document/10994762 *(This paper provides insights into advanced machine learning and deep learning approaches for real estate price prediction, which helped in understanding model design and prediction techniques used in this project.)*

- Park, B., & Bae, J.K. (2015). *"Using Machine Learning Algorithms for Housing Price Prediction."* International Journal of Software Engineering and Its Applications. *(Explains regression and ANN-based prediction methods).*
- Zhou, Y., et al. (2019). *"Real Estate Price Prediction Based on Machine Learning and Deep Learning Techniques."* IEEE Conference. *(Relevant for ANN-based property prediction systems).*
- Justdial. (n.d.). *Local Search and Real Estate Listings Platform.*
  Available at: https://www.justdial.com/
  (Used as a reference for real estate agent information and understanding how location-based services provide property and contact details to users.)
- TensorFlow Keras Guide: https://www.tensorflow.org/guide/keras
  (Used for building and training ANN models).
- Django Documentation: https://docs.djangoproject.com/
  (Used for developing the web application framework).
- Scikit-learn Documentation: https://scikit-learn.org/
  (Used for preprocessing and data handling techniques).
- Kaggle Datasets: https://www.kaggle.com/
  (Source of real estate datasets for training the model).